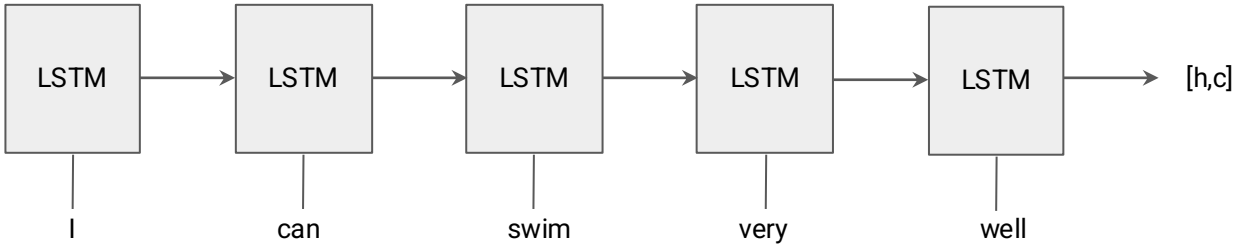
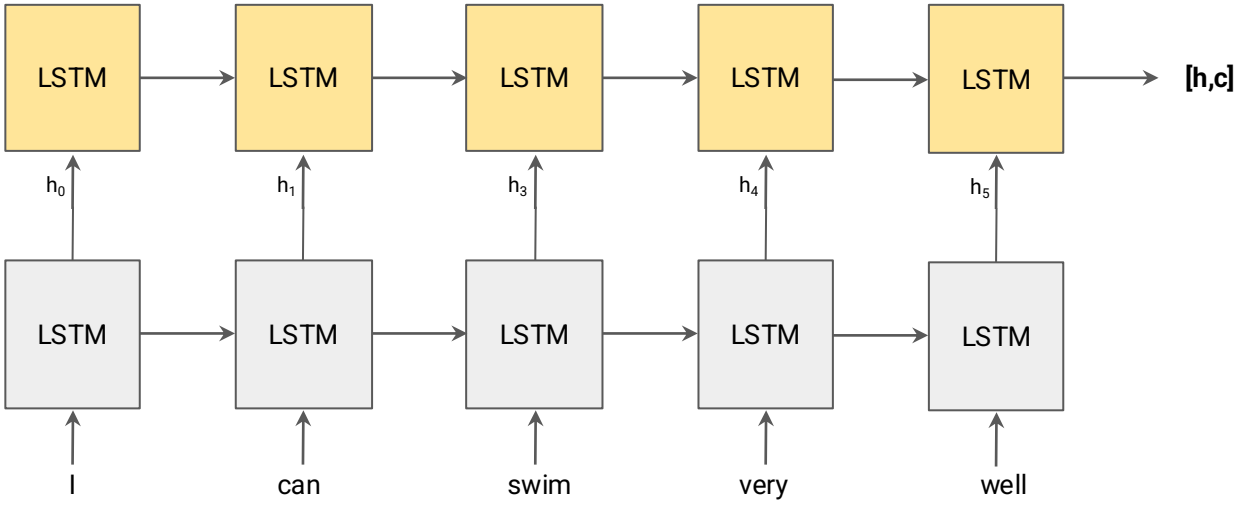
Improving Seq2Seq Model

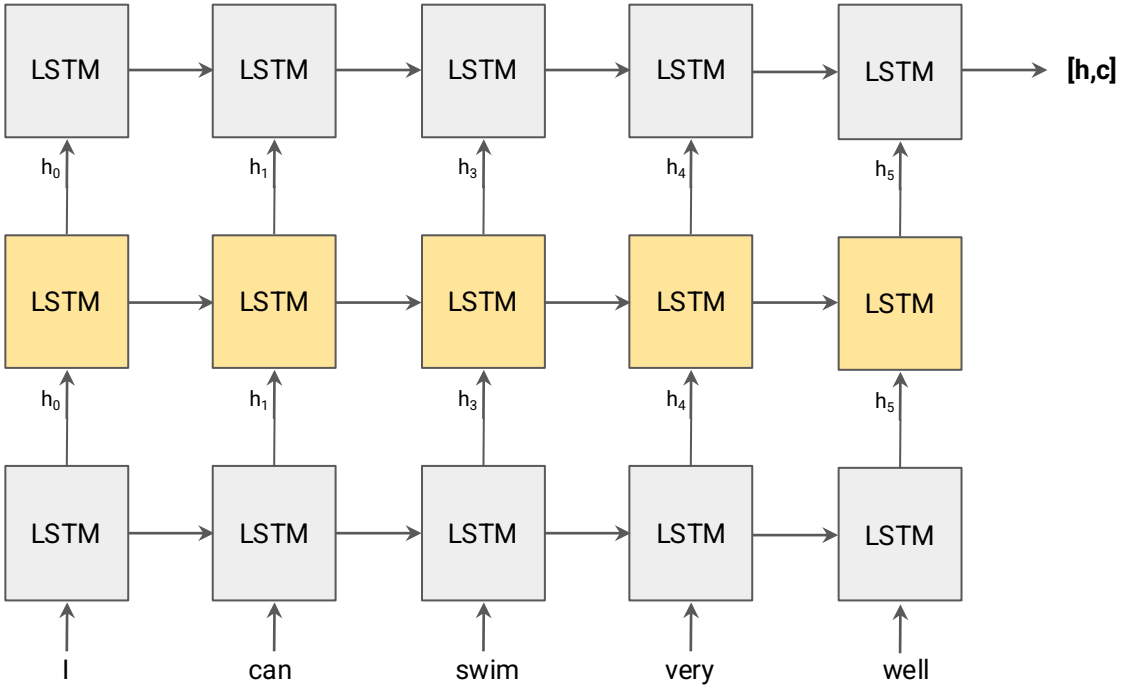


So far...

Stacked LSTM



Two LSTMs stacked over each other .. to learn deeper features

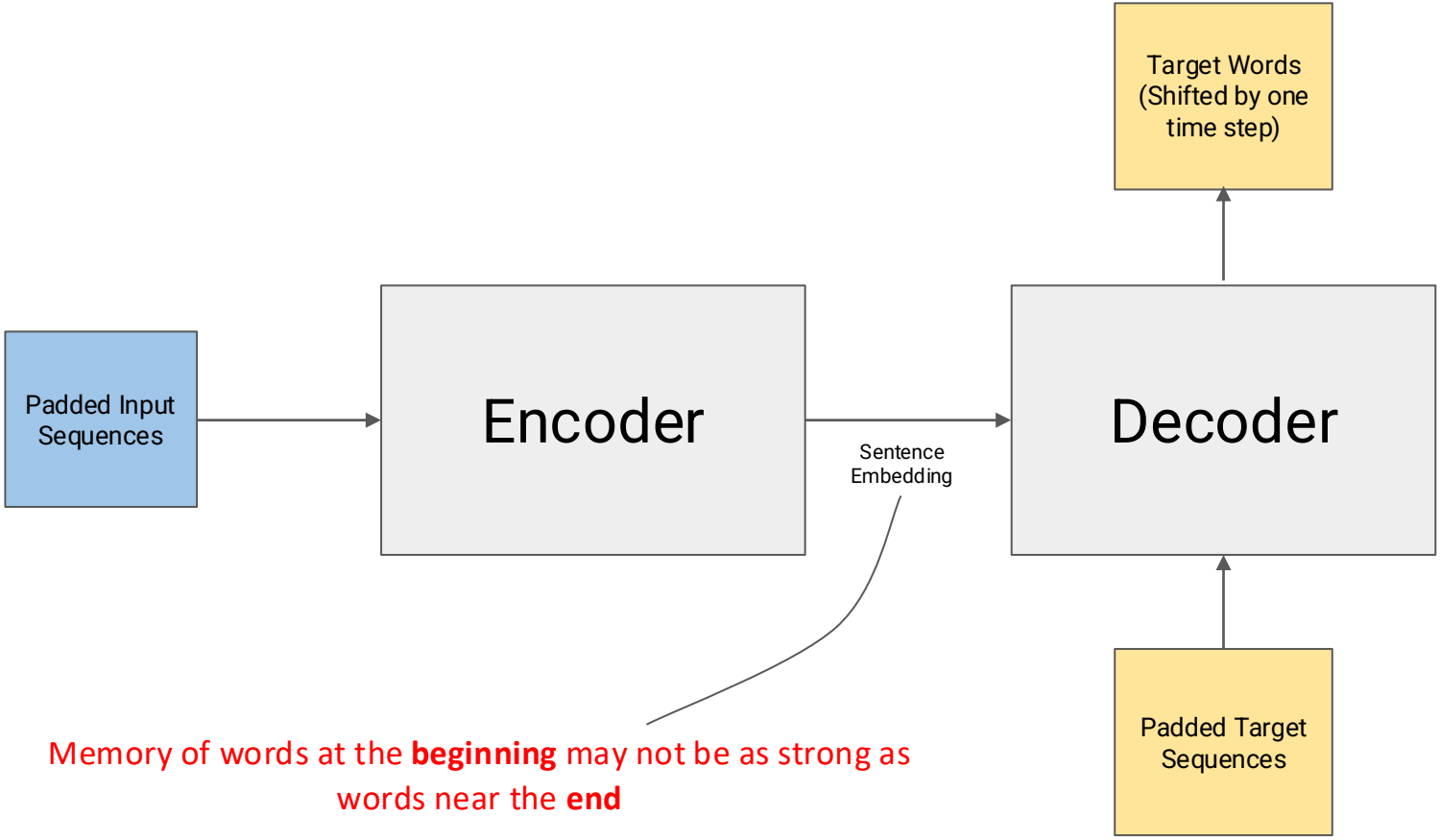


We can stack more and more

```
....
model.add(LSTM(256, return_sequences=True))
model.add(LSTM(256, return_sequences=True))
model.add(LSTM(256, return_sequences=True))
model.add(LSTM(256))
....
```

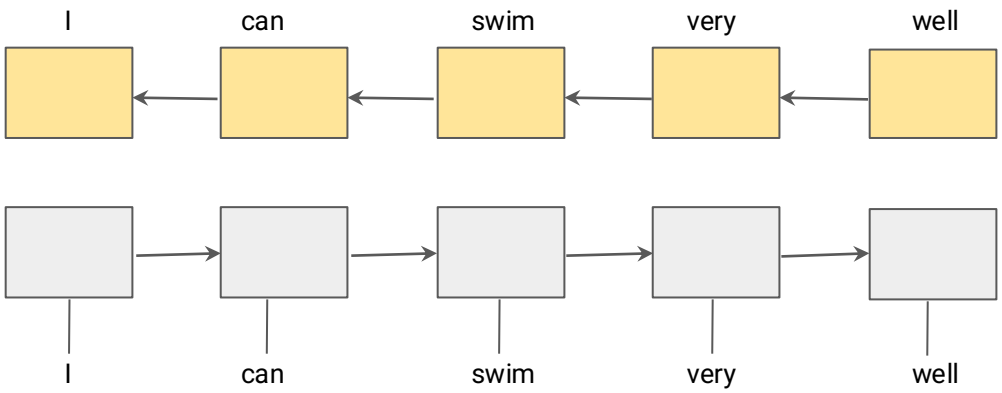
Implementing in Keras - 4 stacked LSTMs

Bidirectional LSTM



Memory of words at the **beginning** may not be as strong as words near the **end**

Bidirectional LSTMs

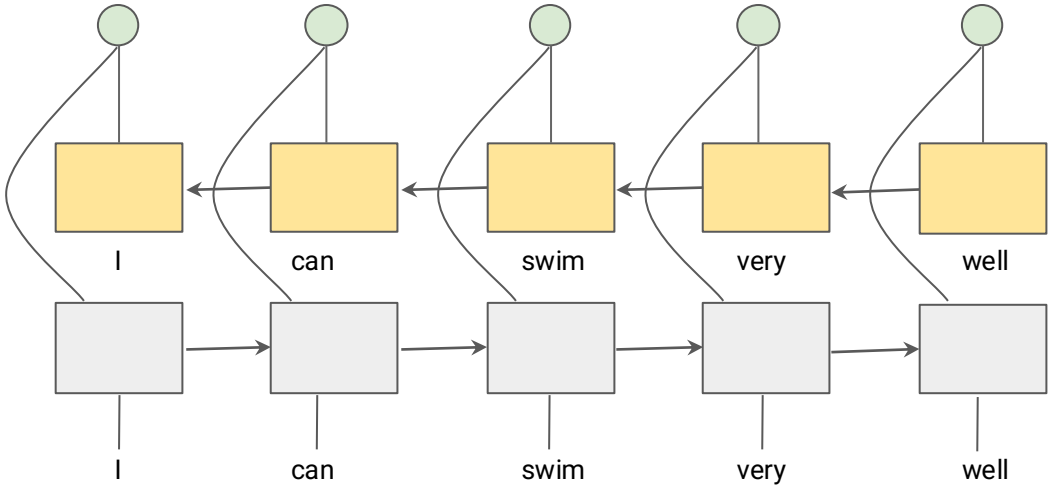


We can use two LSTMs.

- Data for one LSTM gets fed in forward direction
- while for other LSTM, data is fed in reverse direction

... for **better learning**

What is the output Bidirectional LSTM?



Output of both LSTM gets merged . . .

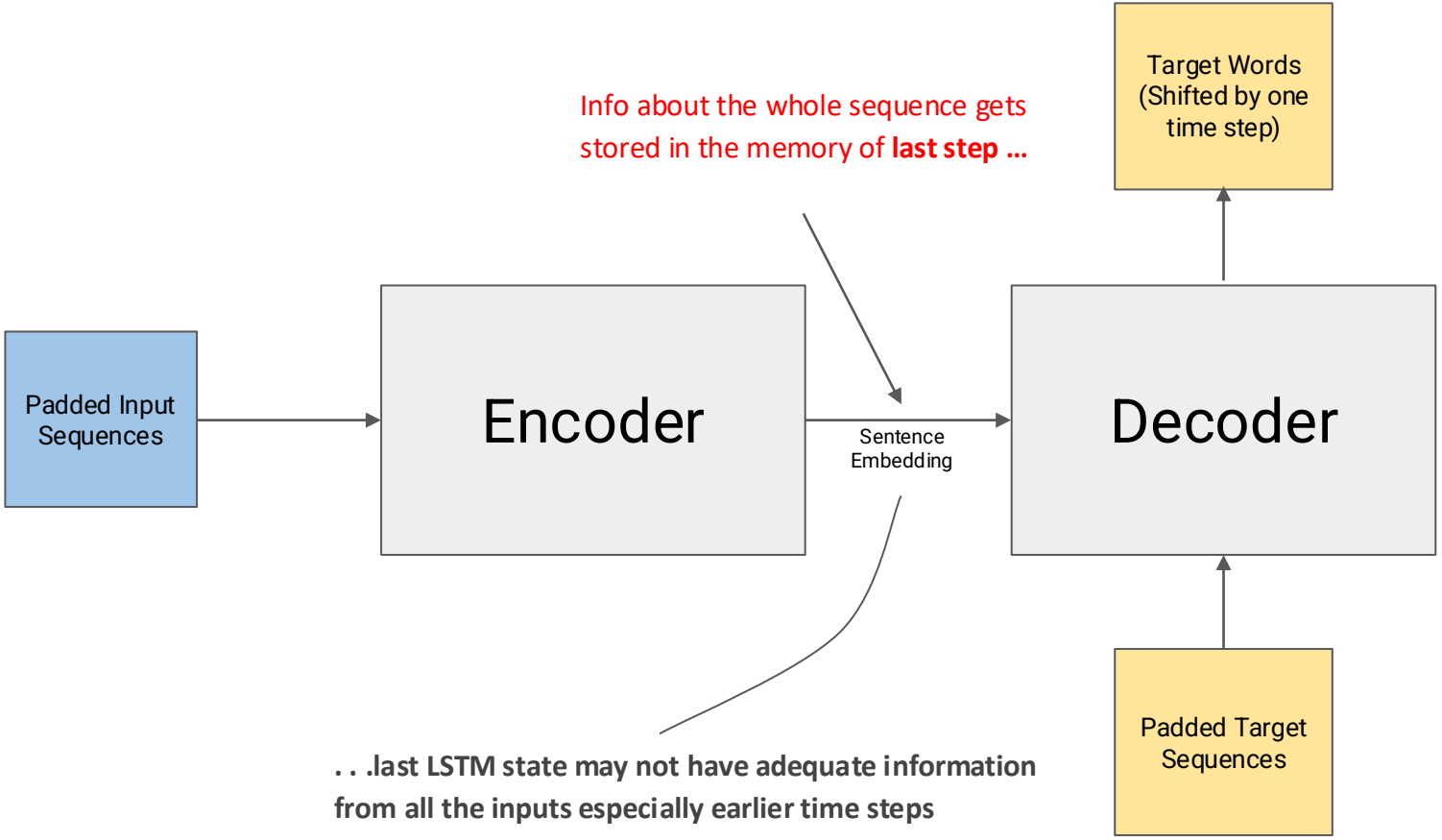
Implementing in Keras - Bidirectional LSTM

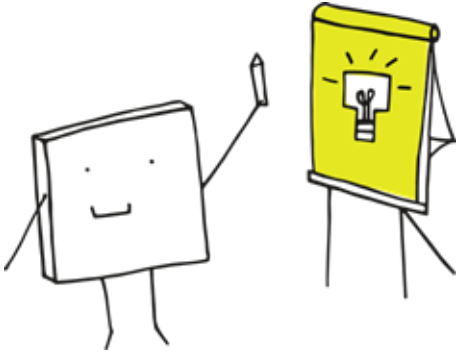
```
model.add(Bidirectional(LSTM(256, return_sequences=True, merge_mode='concat'))
```

merge_mode can be '**concat**', '**sum**' or '**mul**'

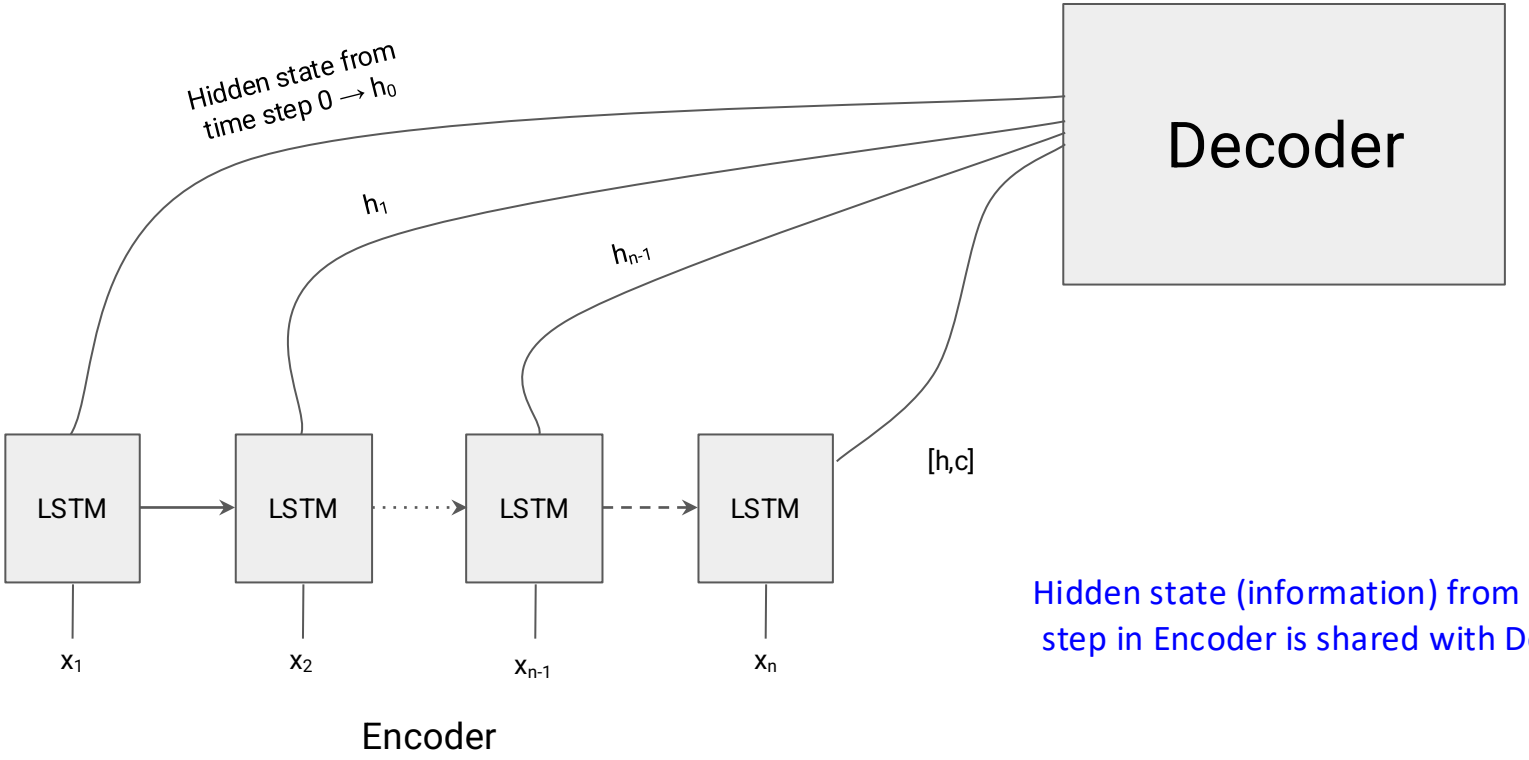
We can try **Deep** Bidirectional LSTM as well

But long sentences can still be problematic :(

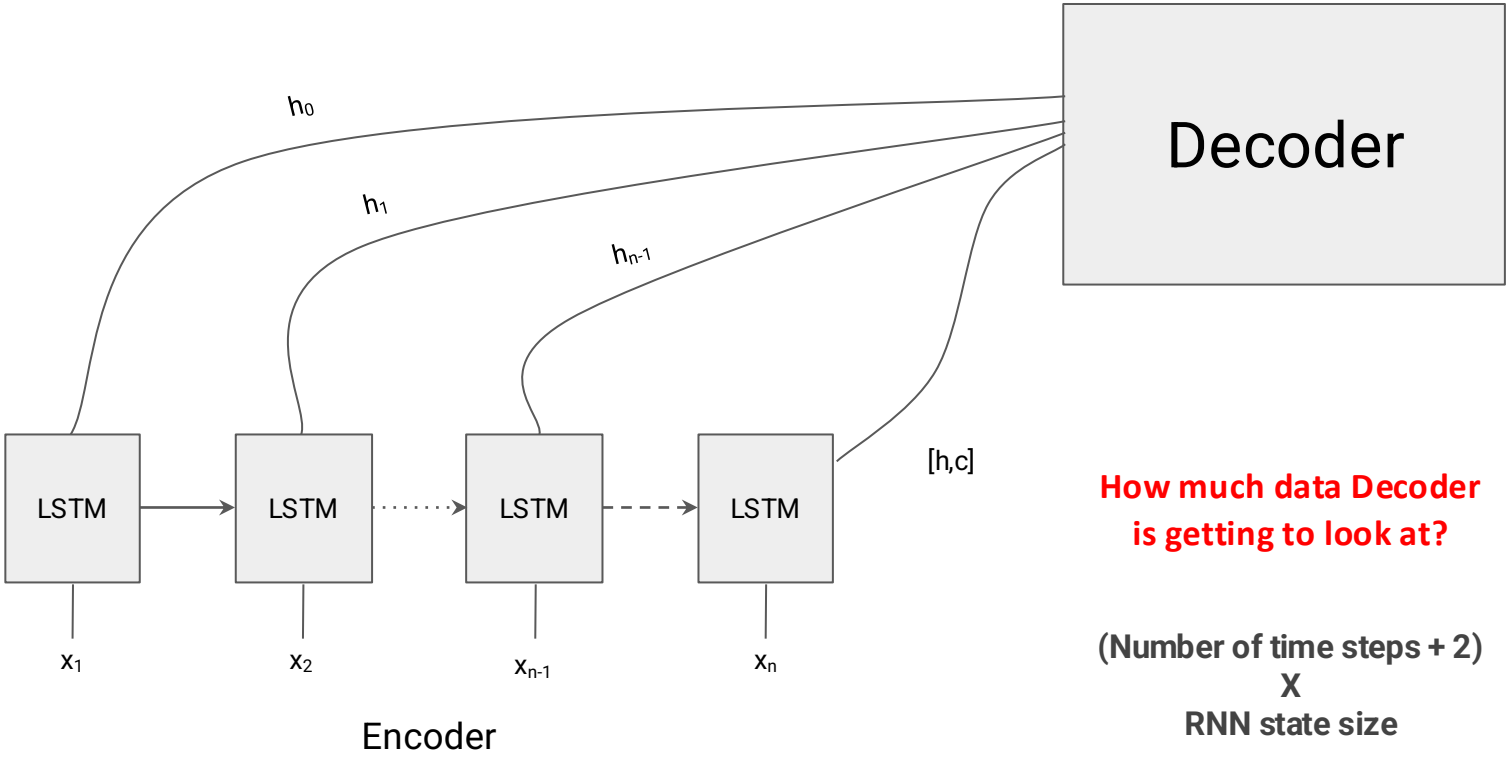

Info about the whole sequence gets stored in the memory of **last step ...**
Target Words (Shifted by one time step)
Padded Input Sequences
Encoder
Sentence Embedding
Decoder
Padded Target Sequences
. . .last LSTM state may not have adequate information from all the inputs especially earlier time steps



What if we provide information from hidden states at all time steps ...



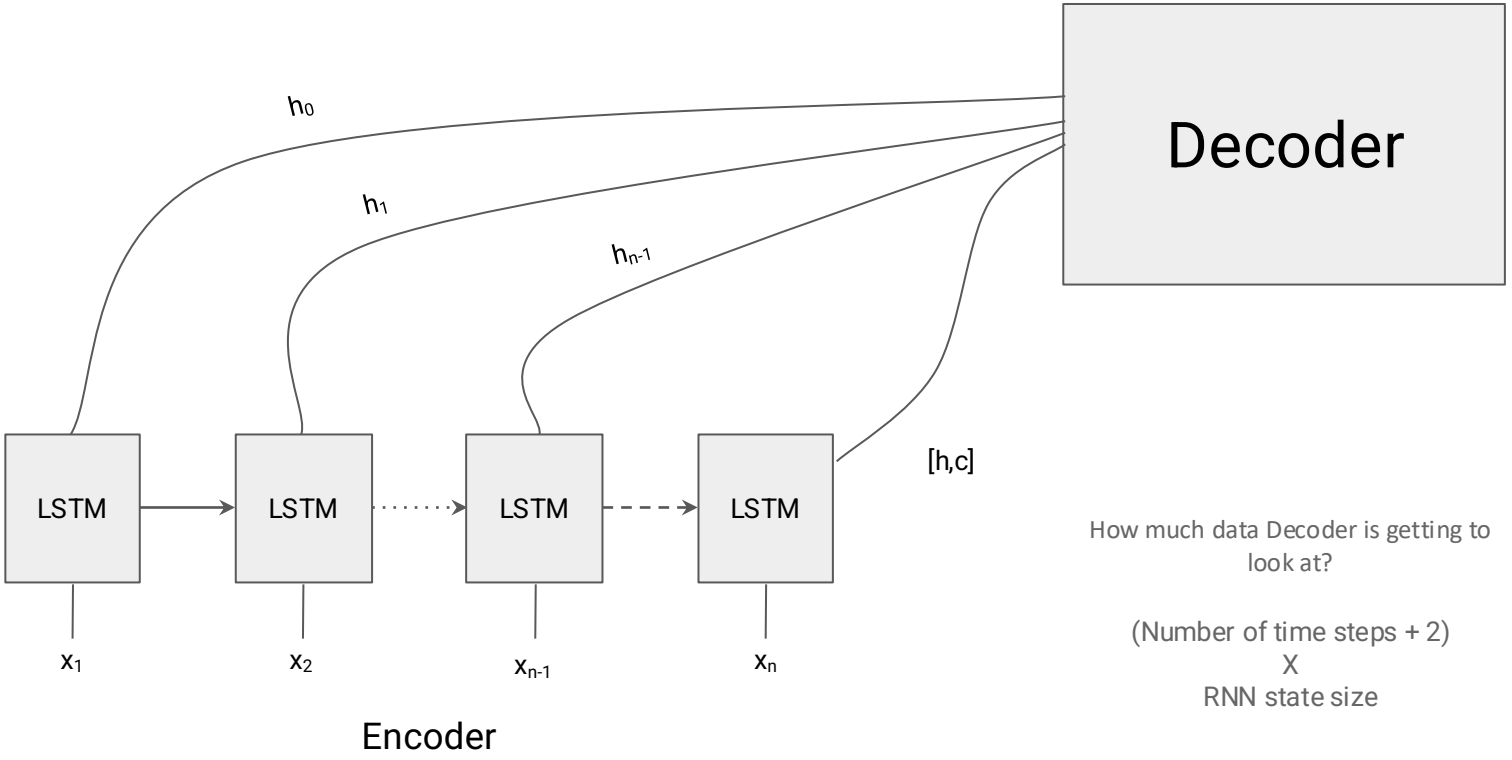
Hidden state (information) from each time step in Encoder is shared with Decoder ...



How much data Decoder is getting to look at?

(Number of time steps + 2)
X
RNN state size

. . . may be decoder is getting too much information :(



How much data Decoder is getting to look at?

(Number of time steps + 2)
X
RNN state size

How can we give <u>only</u> the right information (and not everything)?

Attention

Now what is Attention?

Let’s take an example ...

A Question

Where

did

you

study

The Answer

I

studied

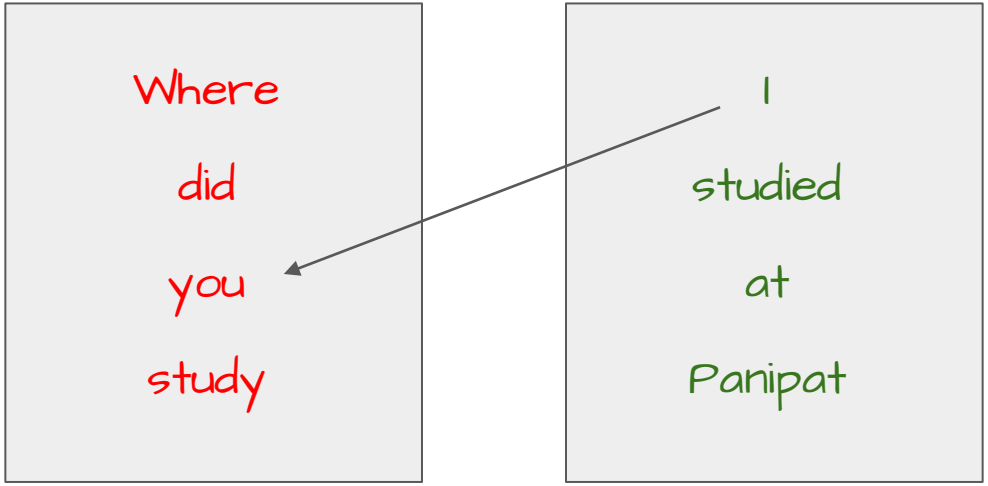
at

Panipat

How did we get different words in the answer?

A Question

An Answer

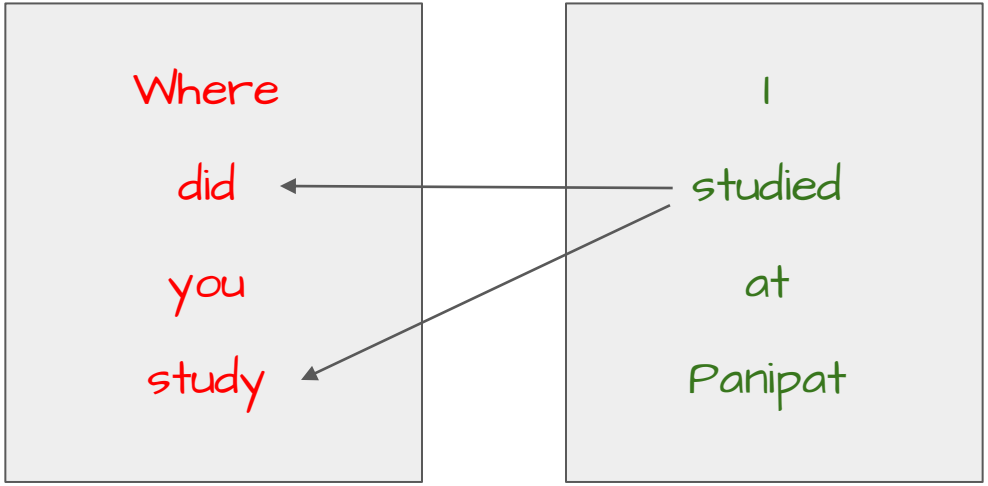


How did we come with 1st word ‘I’?

We paid attention to word ‘you’ in the question ... we can easily ignore other word(s) in the input

A Question

An Answer

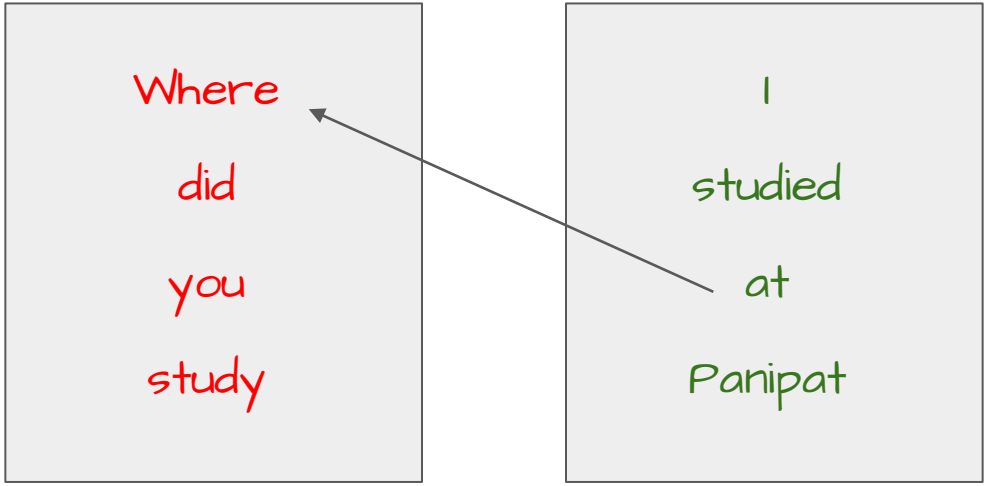


How did get 2nd word 'studied'?

We paid attention to two words 'study' and 'did' in the question ...

A Question

An Answer

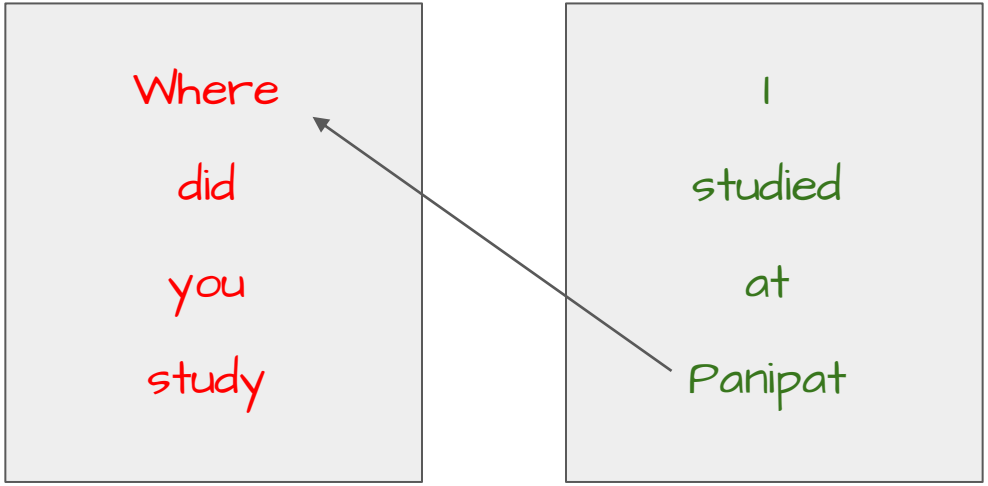


How did get 3rd word 'at'?

We paid attention to the word 'where' in the question ...

A Question

An Answer



How did get 4th word 'Panipat'?

We paid attention to the word 'where' in the question ...

Question

Answer

	Where	did	you	study
I	No	No	**Yes**	No
studied	No	**Yes**	No	**Yes**
at	**Yes**	No	No	No
Panipat	**Yes**	No	No	No

Attention Matrix

Question

Answer

	Where	did	you	study
I	0.0	0.0	**1.0**	0.0
studied	0.0	**0.3**	0.0	**0.7**
at	**1.0**	0.0	0.0	0.0
Panipat	**1.0**	0.0	0.0	0.0

Attention Matrix

(In numbers)

Question

Answer

	Where	did	you	study
I	0.0	0.0	1.0	0.0
studied	0.0	0.3	0.0	0.7
at	1.0	0.0	0.0	0.0
Panipat	1.0	0.0	0.0	0.0

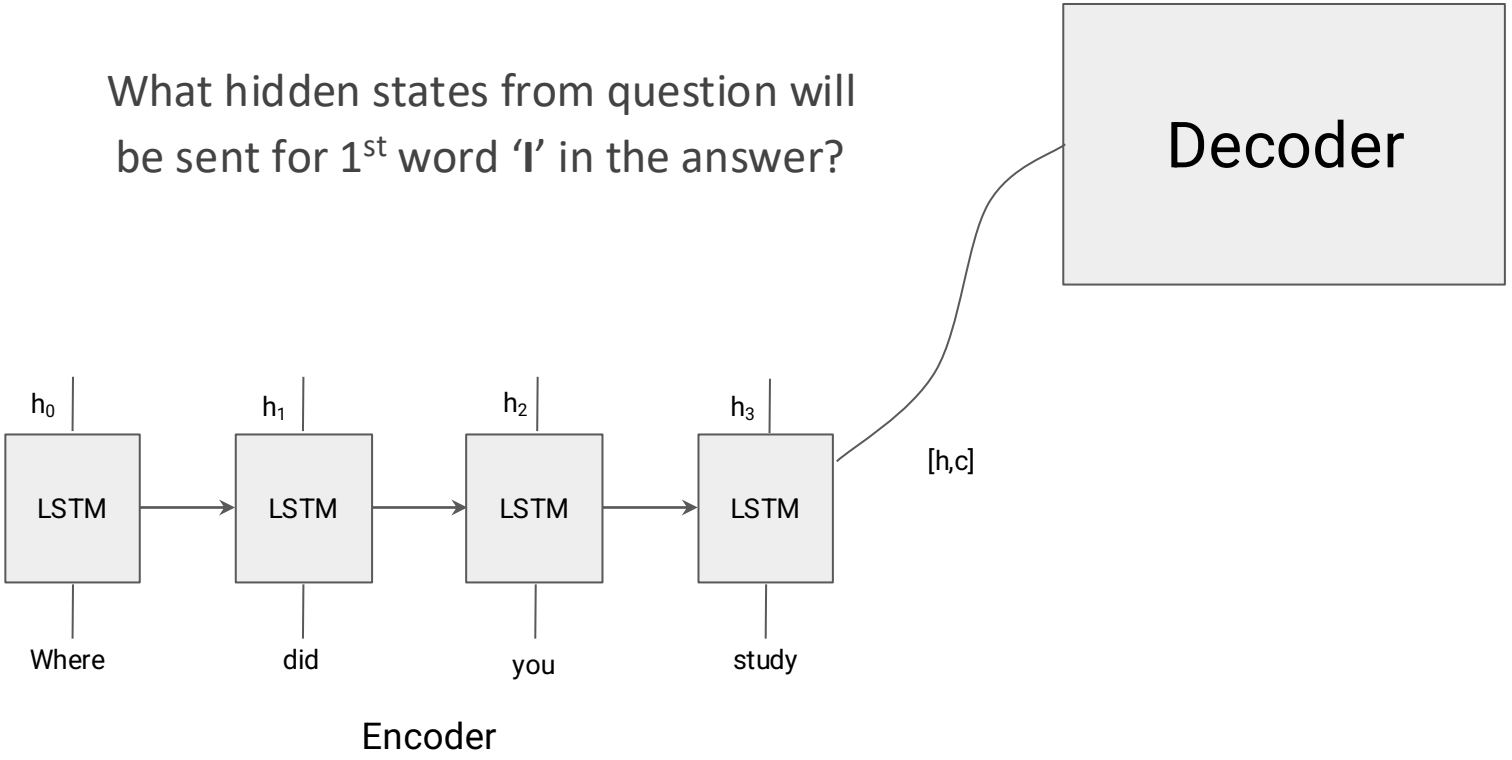
‘Studied’ paid more attention on ‘study’
and smaller attention on ‘did’.

Attention Matrix

(In numbers)

How would Attention look in Seq2Seq model?

What hidden states from question will be sent for 1st word ‘I’ in the answer?

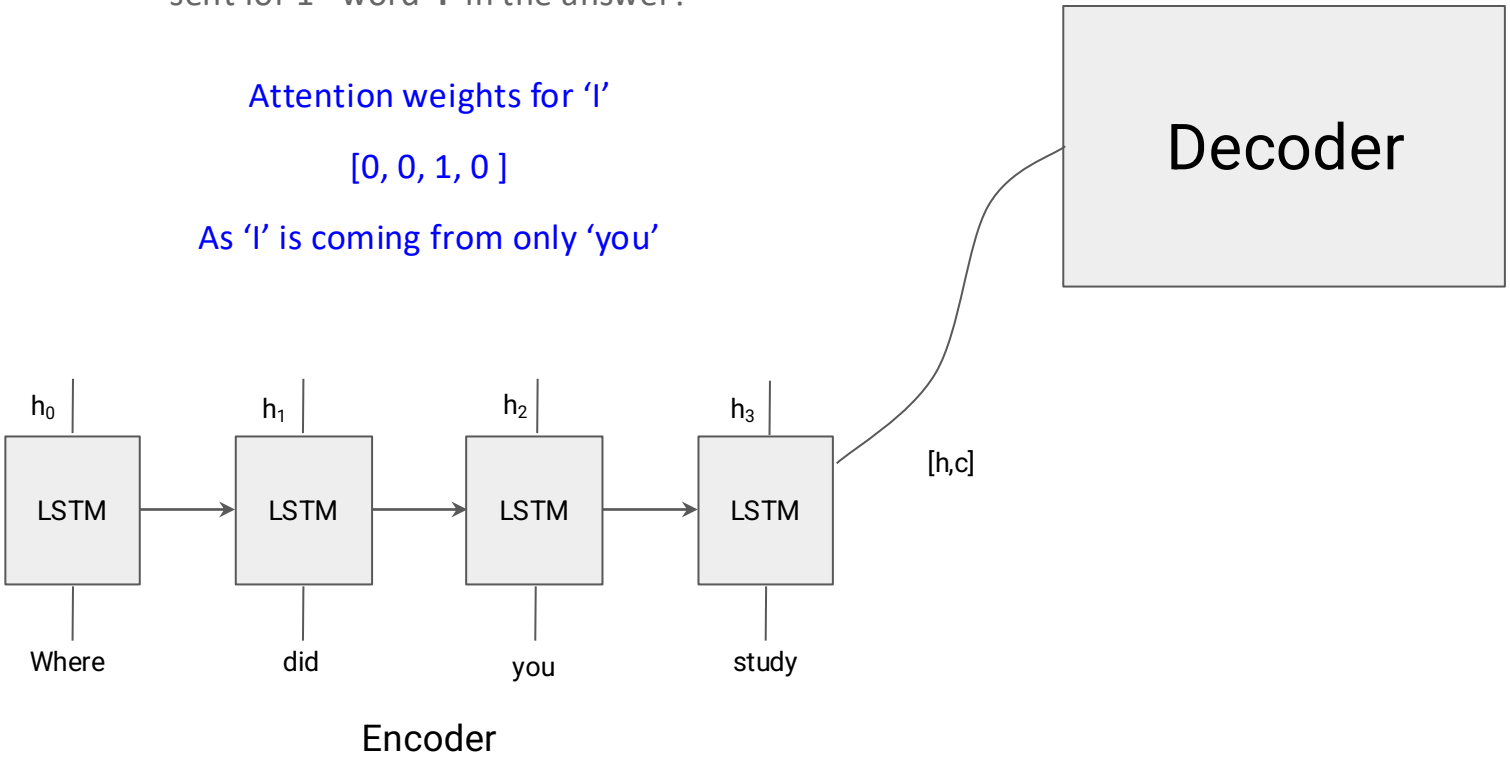


What hidden states from question will be sent for 1st word '**I**' in the answer?

Attention weights for 'I'

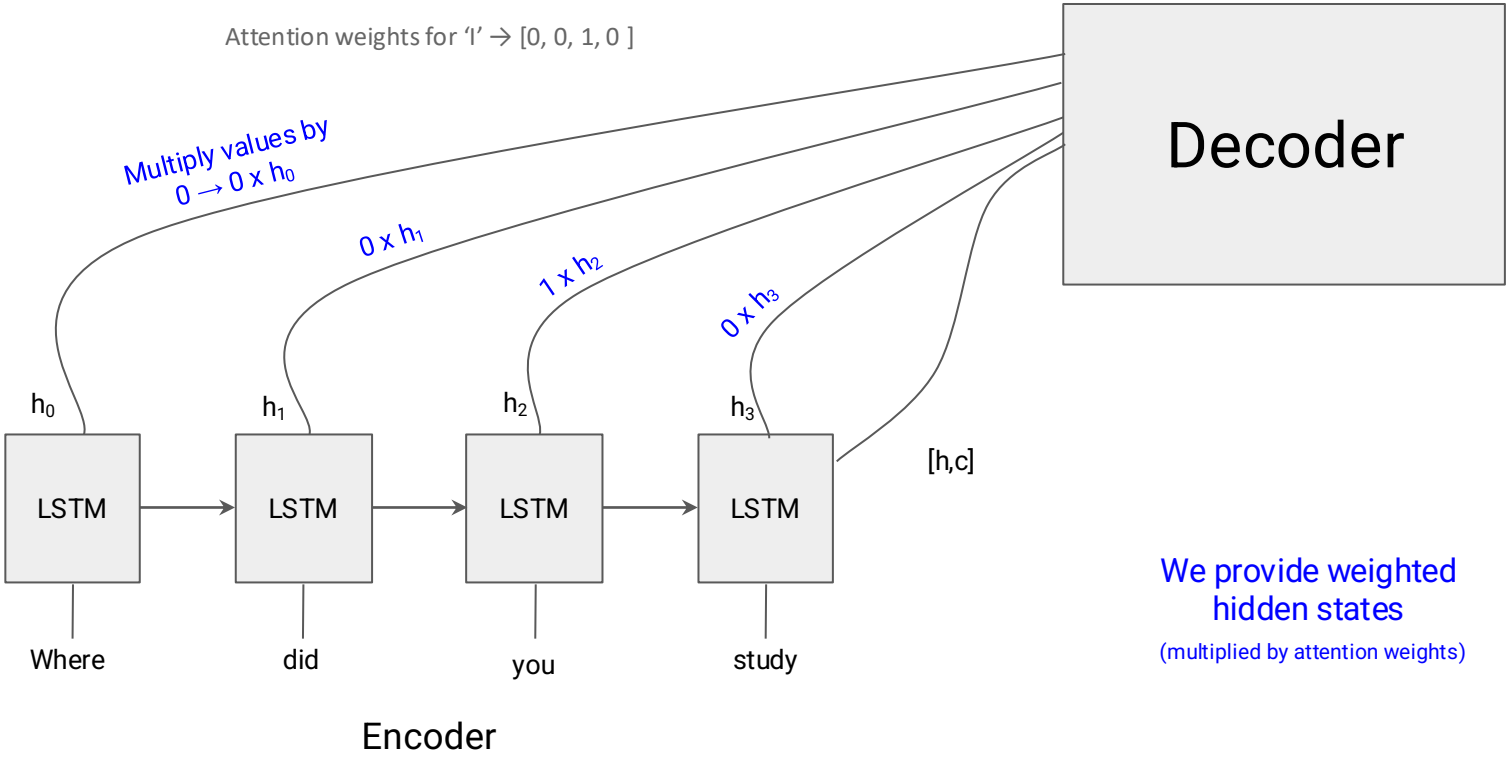
[0, 0, 1, 0]

As 'I' is coming from only 'you'



What hidden states from question will be sent for 1st word ‘I’ in the answer?

Attention weights for ‘I’ → [0, 0, 1, 0]

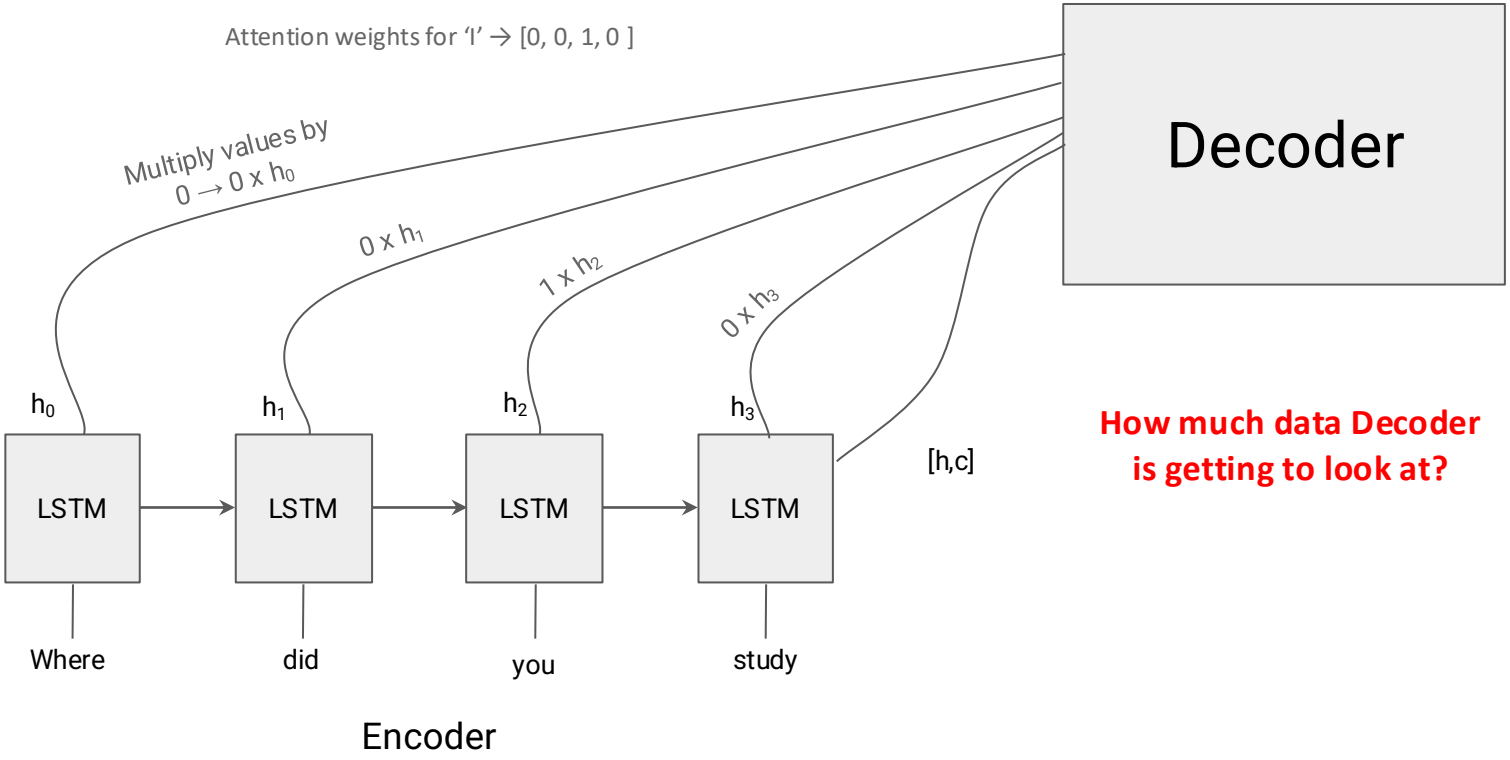


We provide weighted hidden states

(multiplied by attention weights)

What hidden states from question will be sent for 1st word ‘**I**’ in the answer?

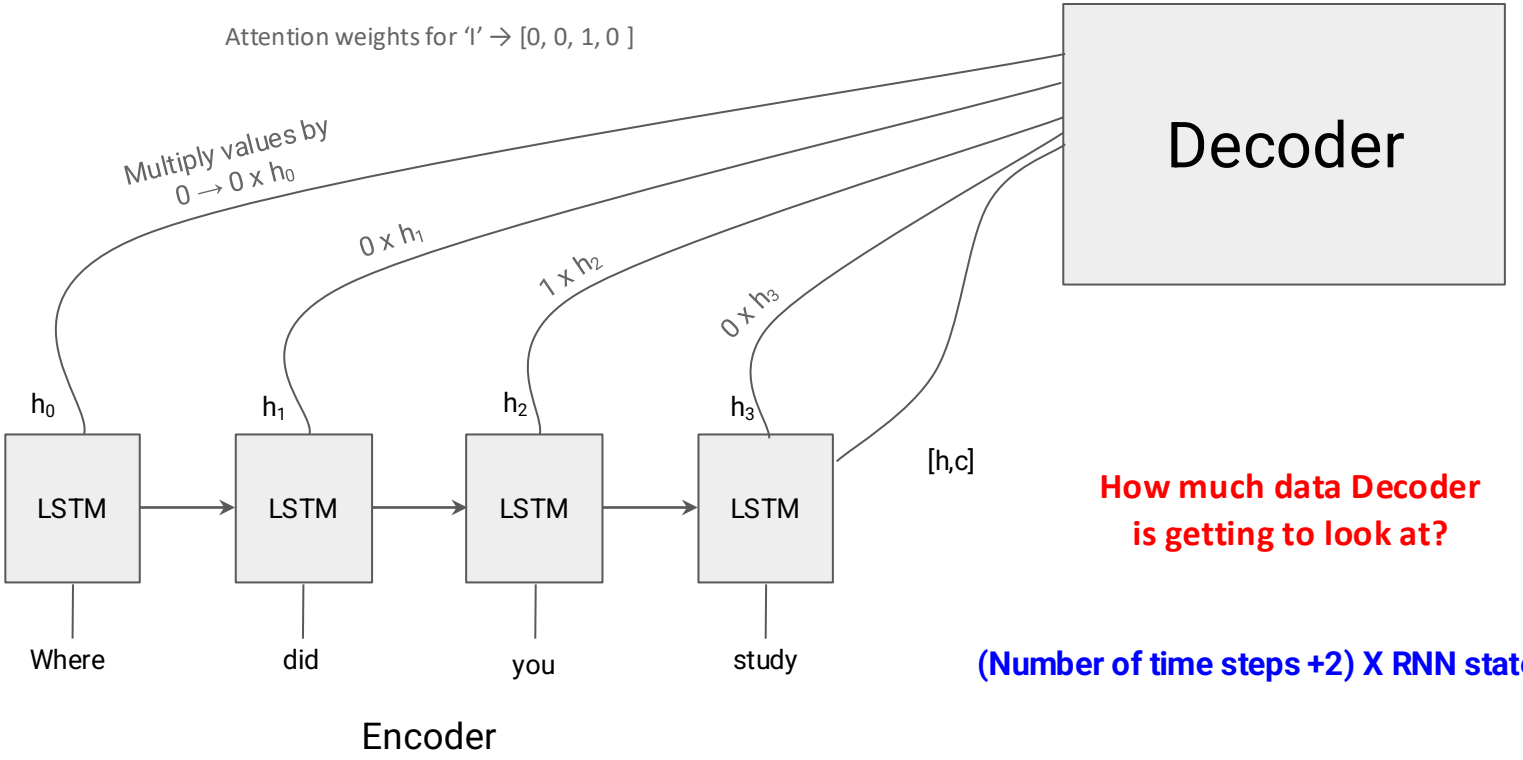
Attention weights for ‘I’ → [0, 0, 1, 0]



How much data Decoder is getting to look at?

What hidden states from question will be sent for 1st word ‘I’ in the answer?

Attention weights for ‘I’ → [0, 0, 1, 0]



How much data Decoder is getting to look at?

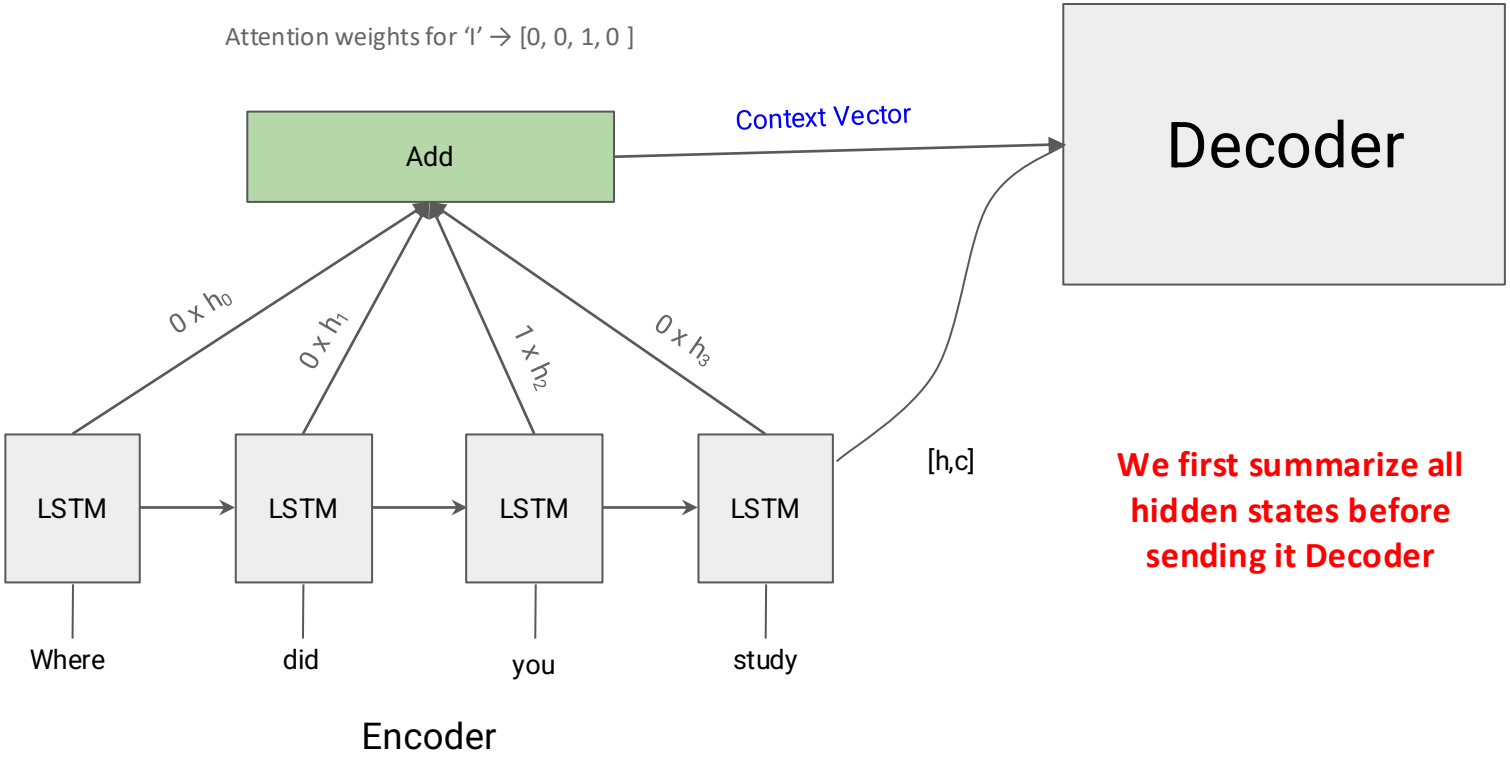
(Number of time steps +2) X RNN state size

. . . still the same amount of numbers as earlier :(

How do we provide ‘only’ the right info with less data to Decoder?

What hidden states from question will be sent for 1st word ‘**I**’ in the answer?

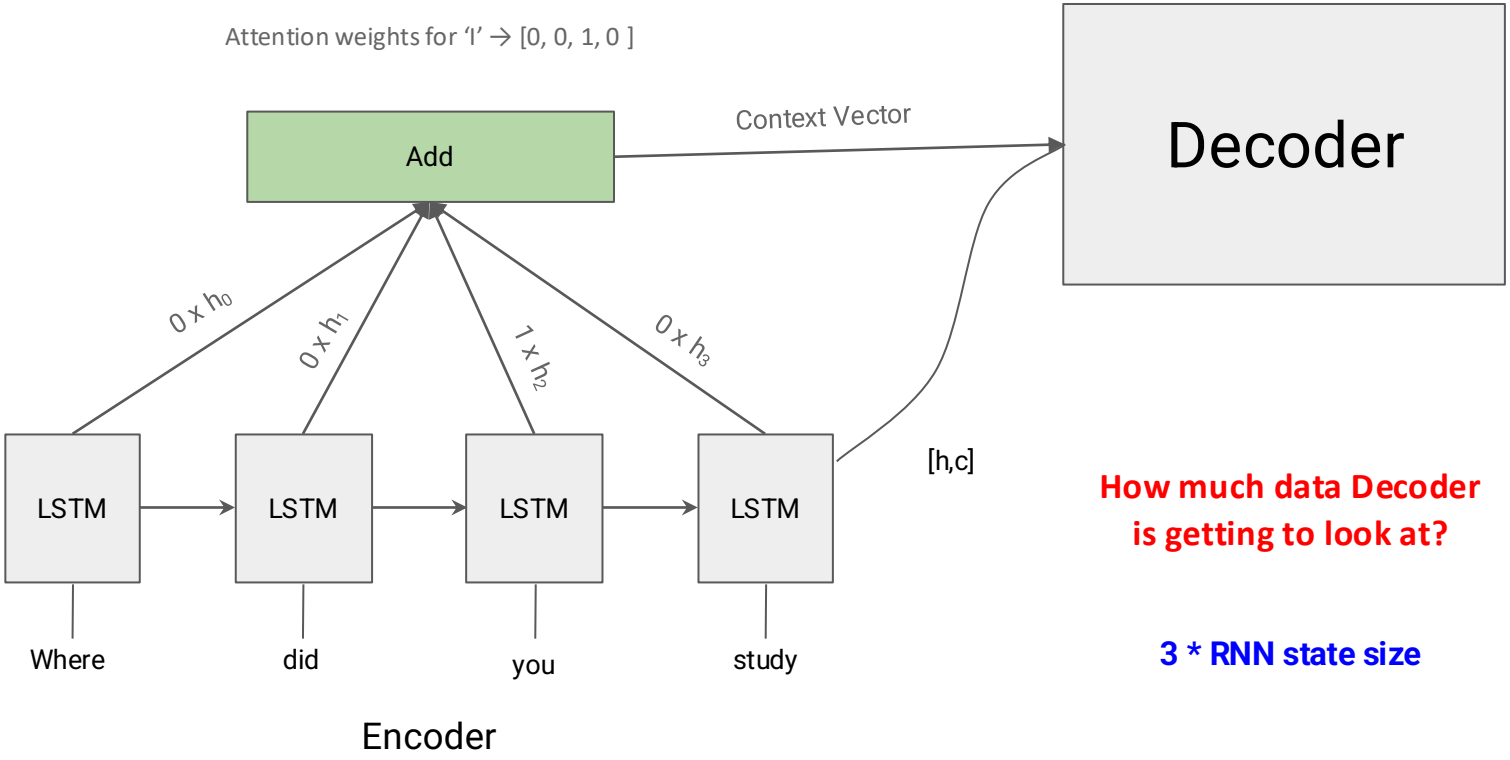
Attention weights for ‘I’ → [0, 0, 1, 0]



We first summarize all hidden states before sending it Decoder

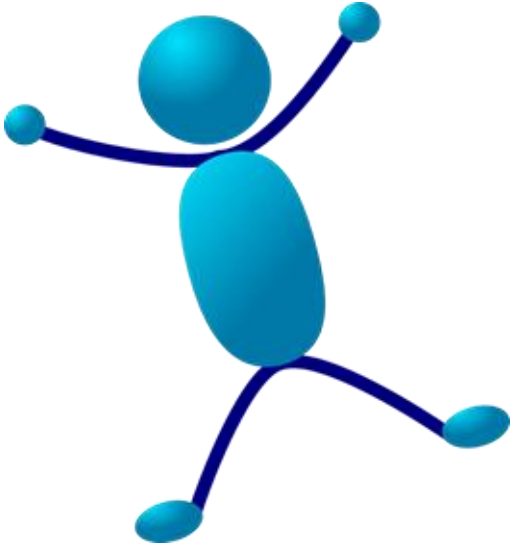
What hidden states from question will be sent for 1st word ‘I’ in the answer?

Attention weights for ‘I’ → [0, 0, 1, 0]



How much data Decoder is getting to look at?

3 * RNN state size

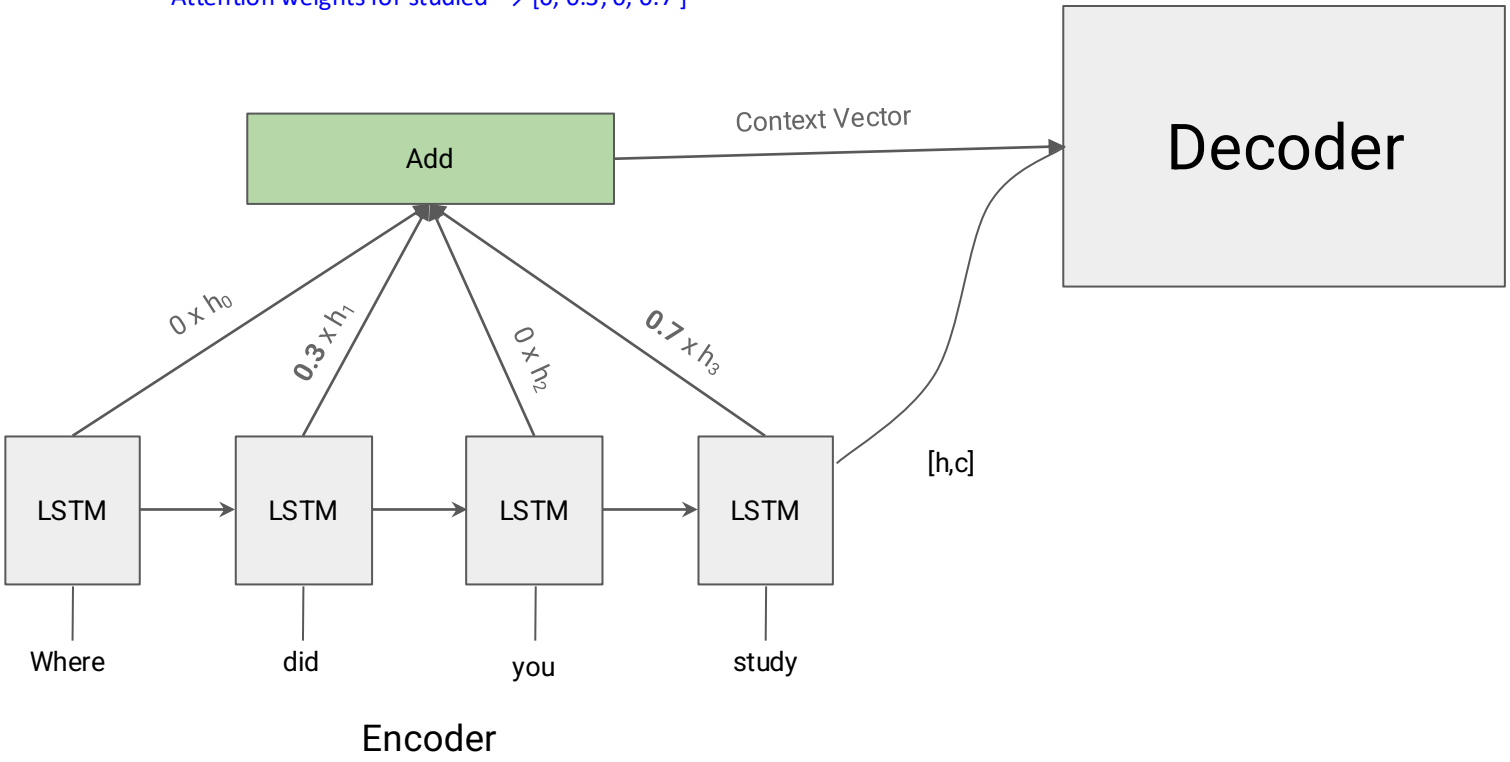


Yay!

We are providing ‘only’ relevant information with less amount of numbers!!!

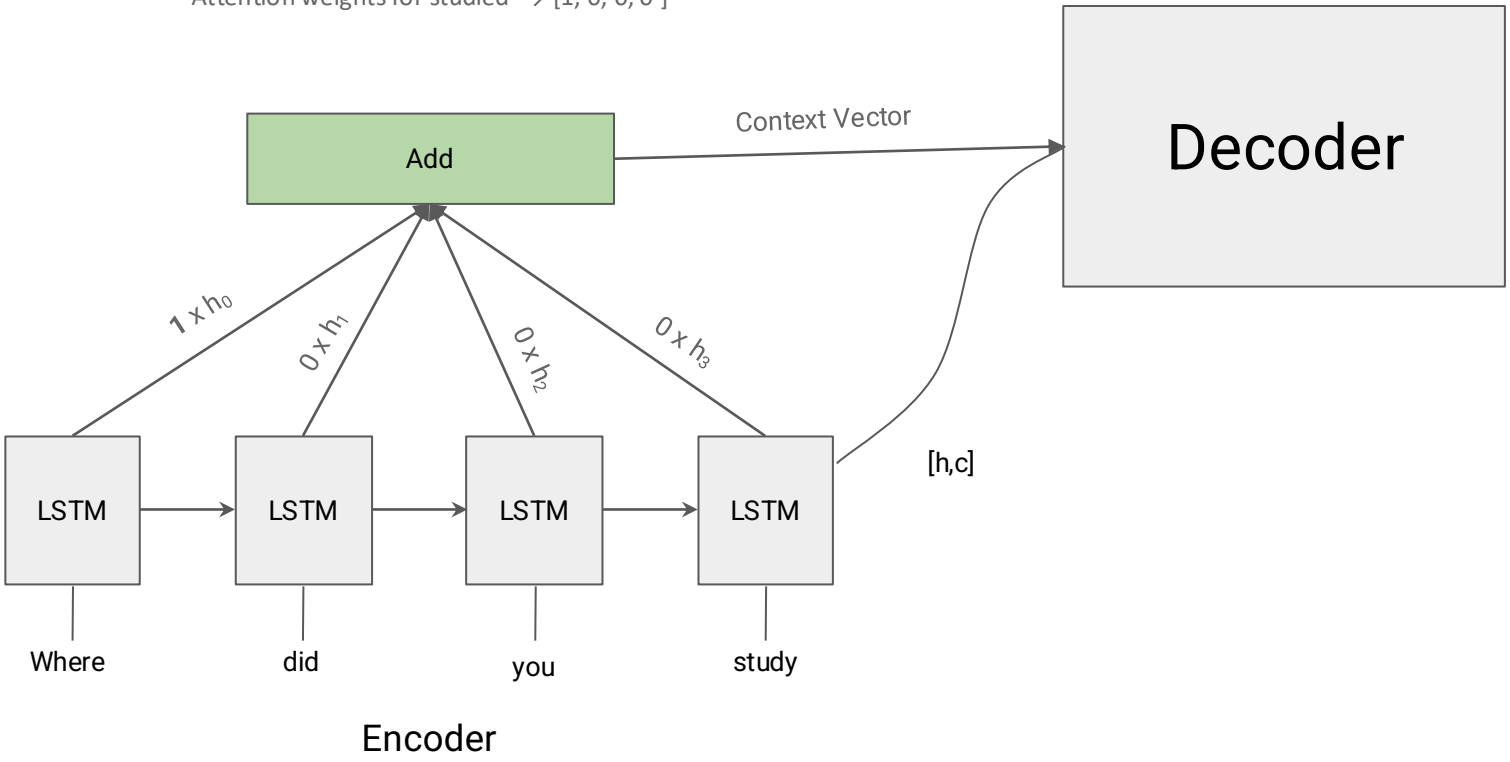
Context vector for 2nd word '**studied**'

Attention weights for studied' → [0, 0.3, 0, 0.7]



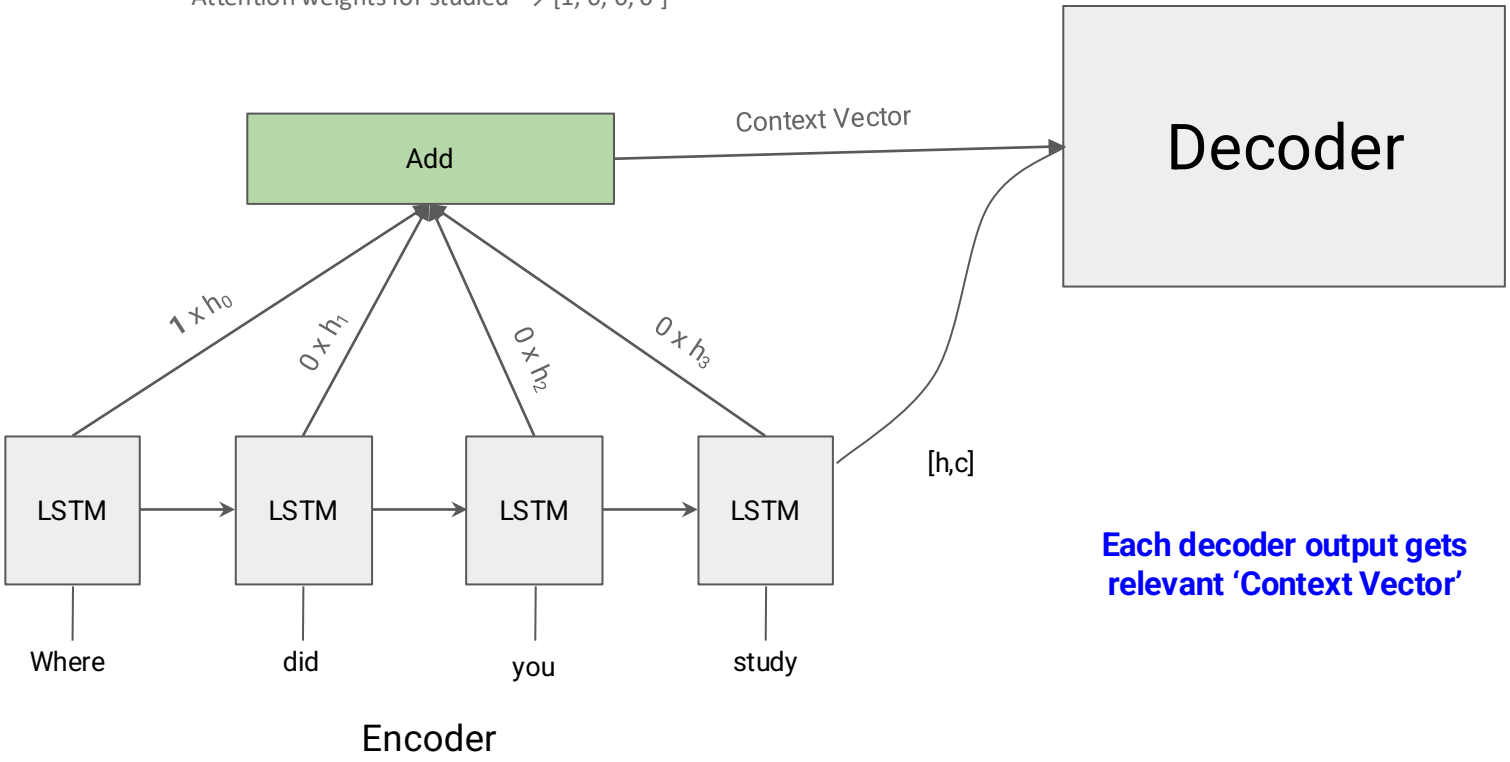
Context vector for 3rd word '**at**'

Attention weights for studied' → [1, 0, 0, 0]



Context vector for 4th word '**Panipat**'

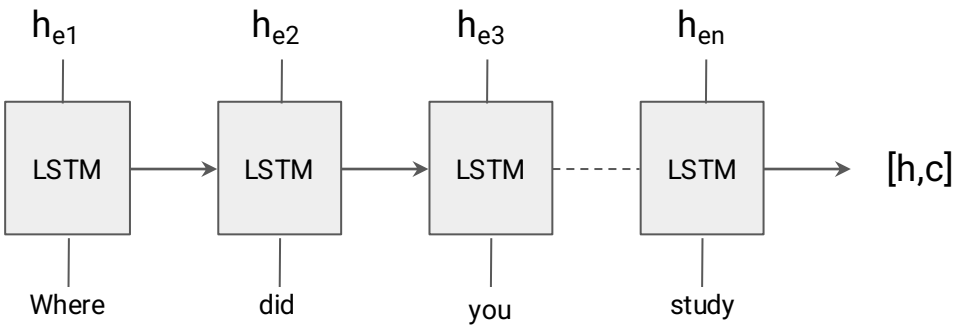
Attention weights for studied' → [1, 0, 0, 0]



Each decoder output gets relevant 'Context Vector'

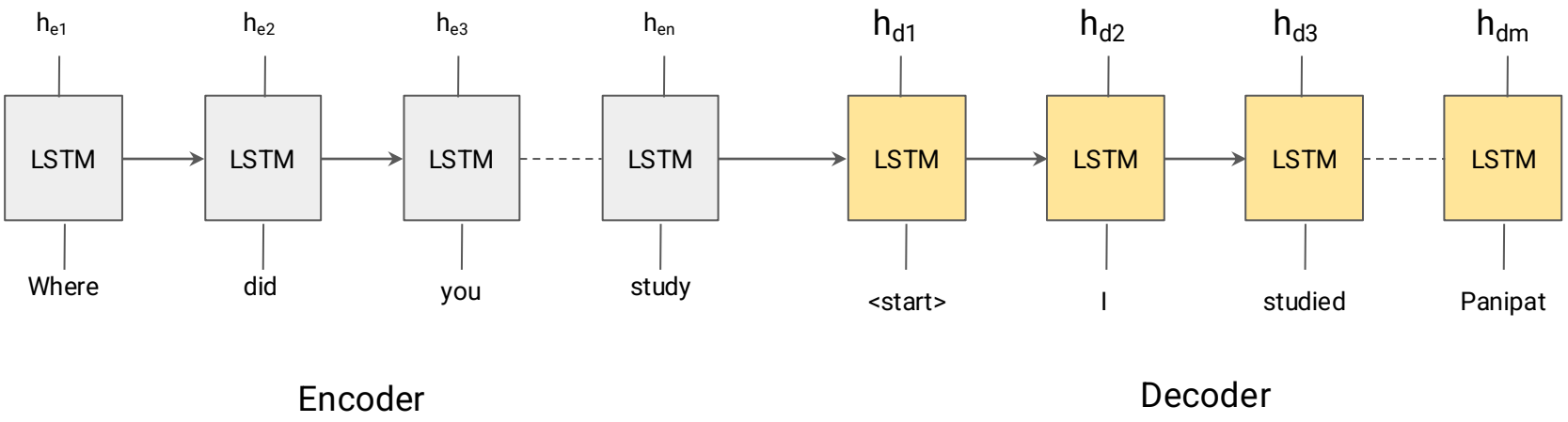
How do we get **Attention weights** for each Decoder output?

Encoder LSTM generates hidden states at each step and also provides sentence embedding



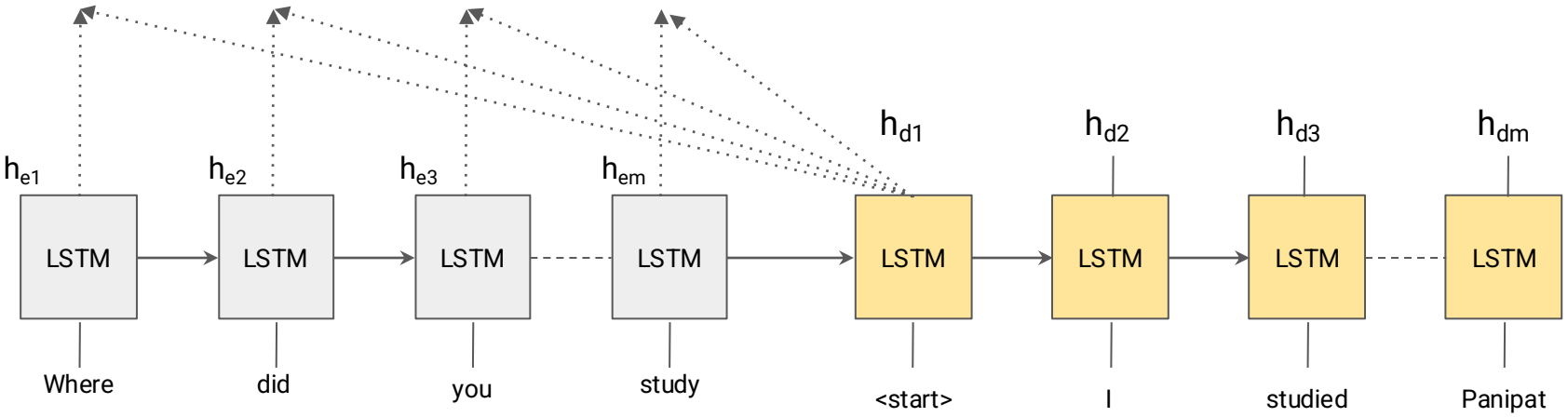
Encoder

Decoder takes sentence embedding and generates hidden state at each step



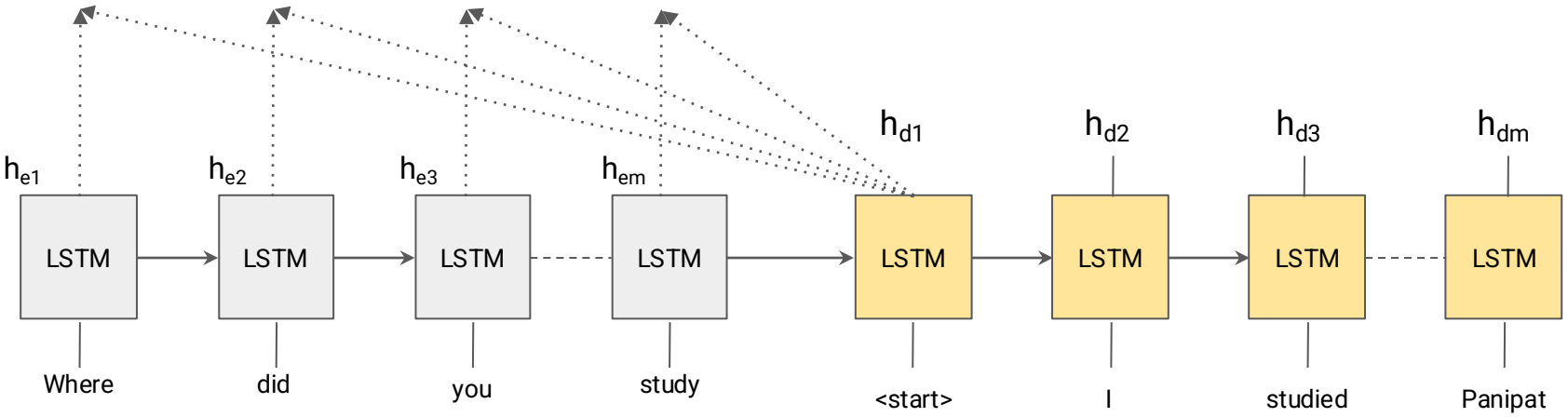
Calculating attention weights for 1st decoder output

Align all encoder states h_e with 1st decoder hidden state h_{d1}



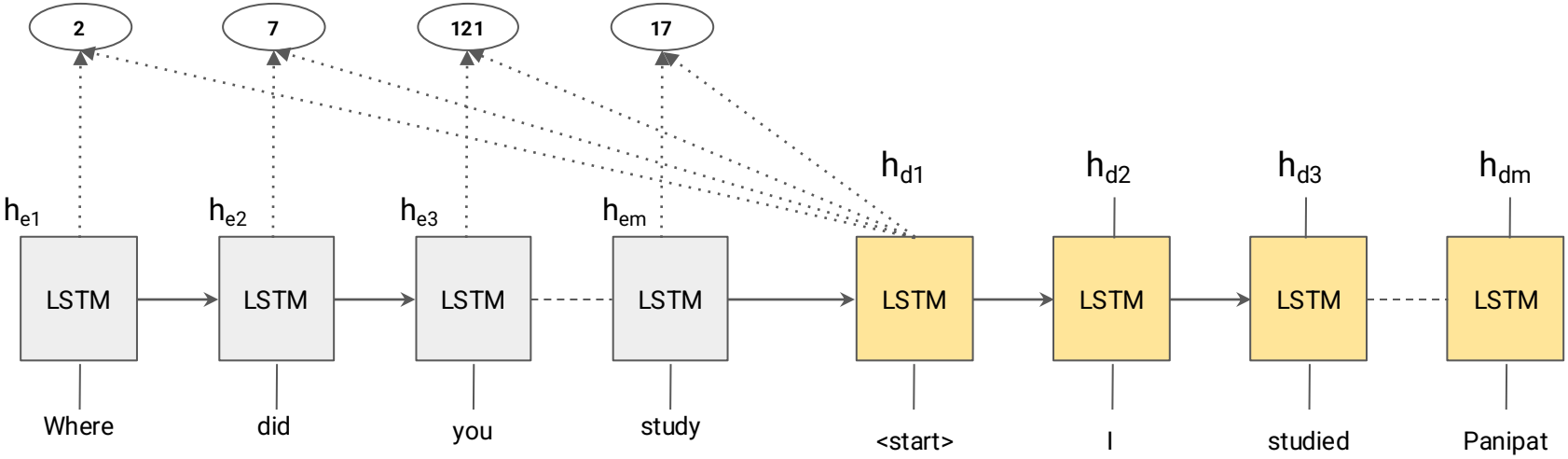
How do we align hidden states?

We do a '**dot**' product between each encoder states h_e with 1st decoder hidden state h_{d1}



For each input word, we will get

one number

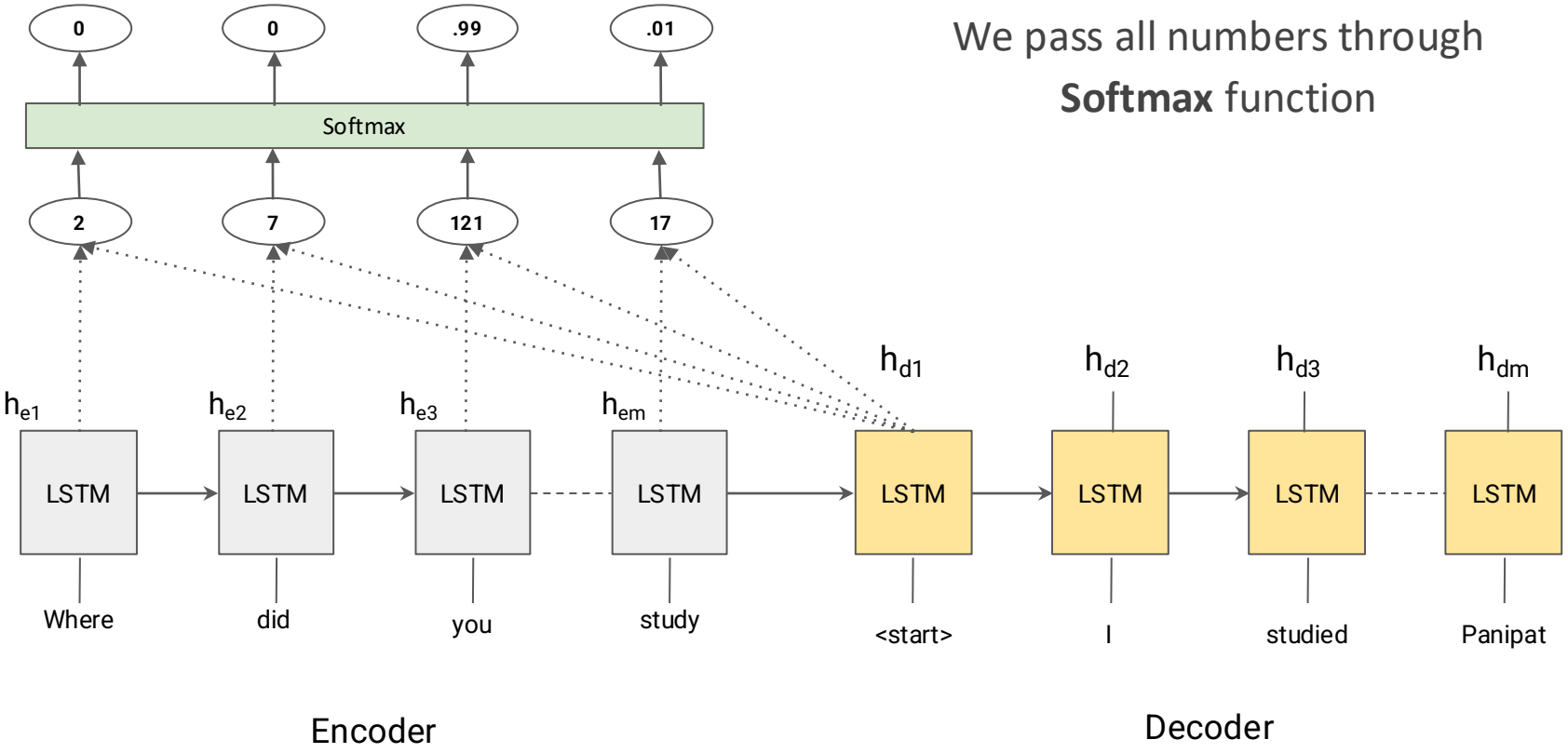


Encoder

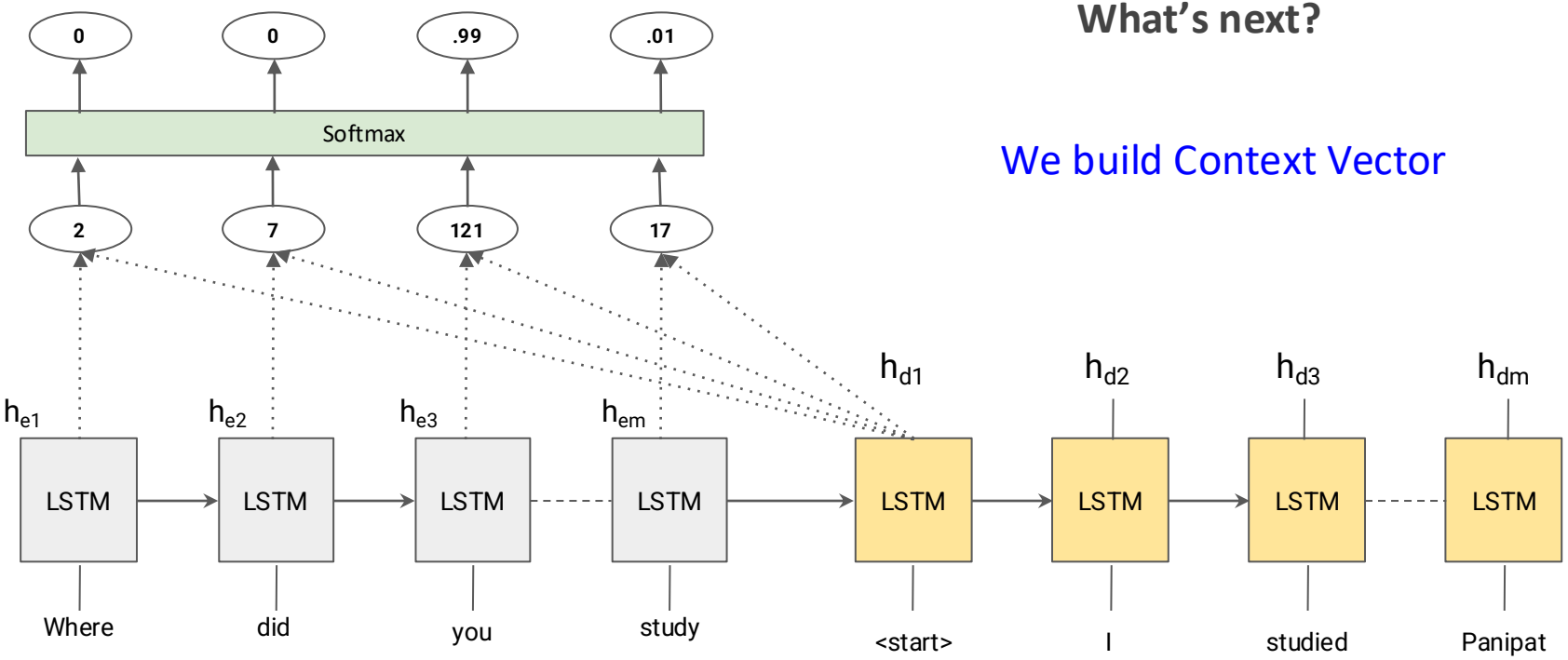
Decoder

Attention weights to generate I

We pass all numbers through **Softmax** function



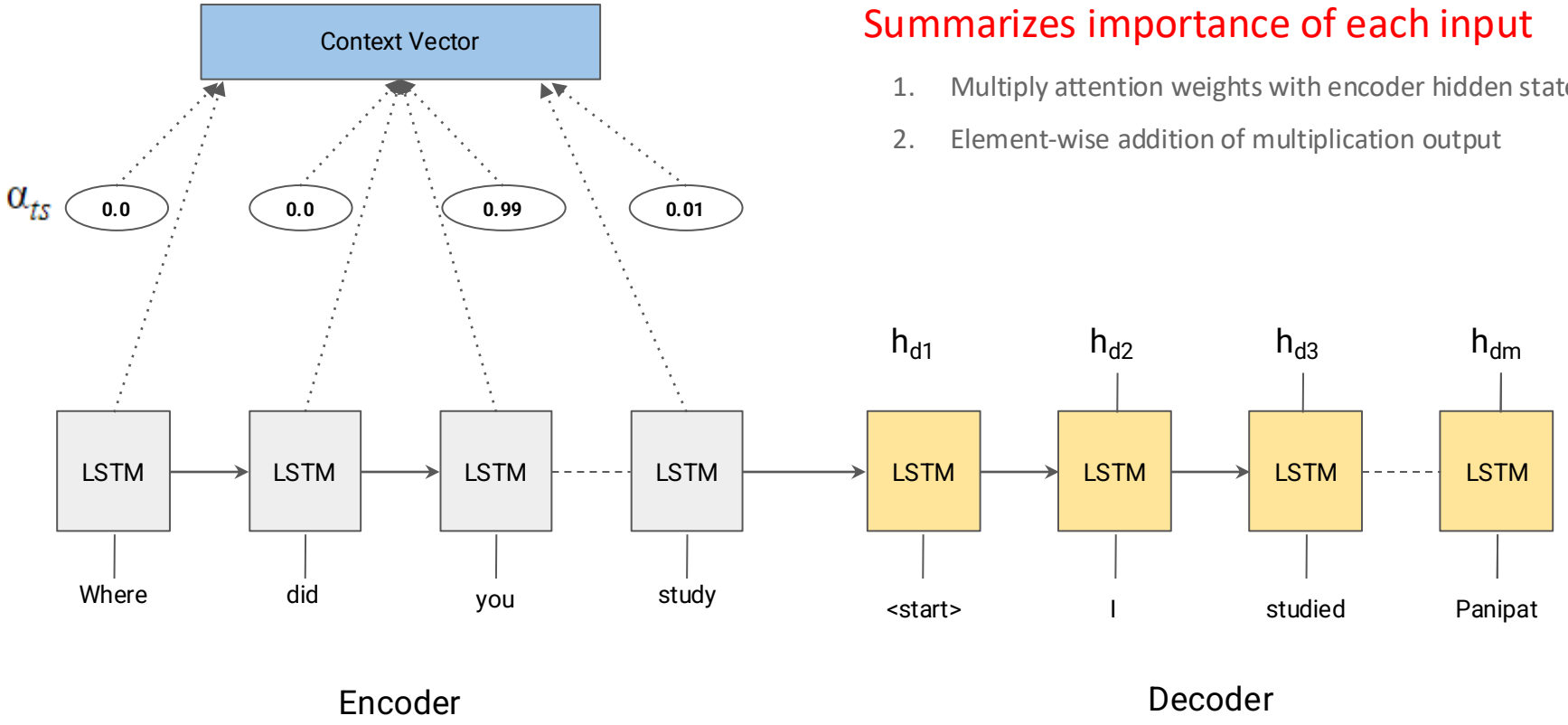
Attention weights to generate I



What's next?

We build Context Vector

$$c_t = \sum \alpha_{ts} h_e$$



Summarizes importance of each input

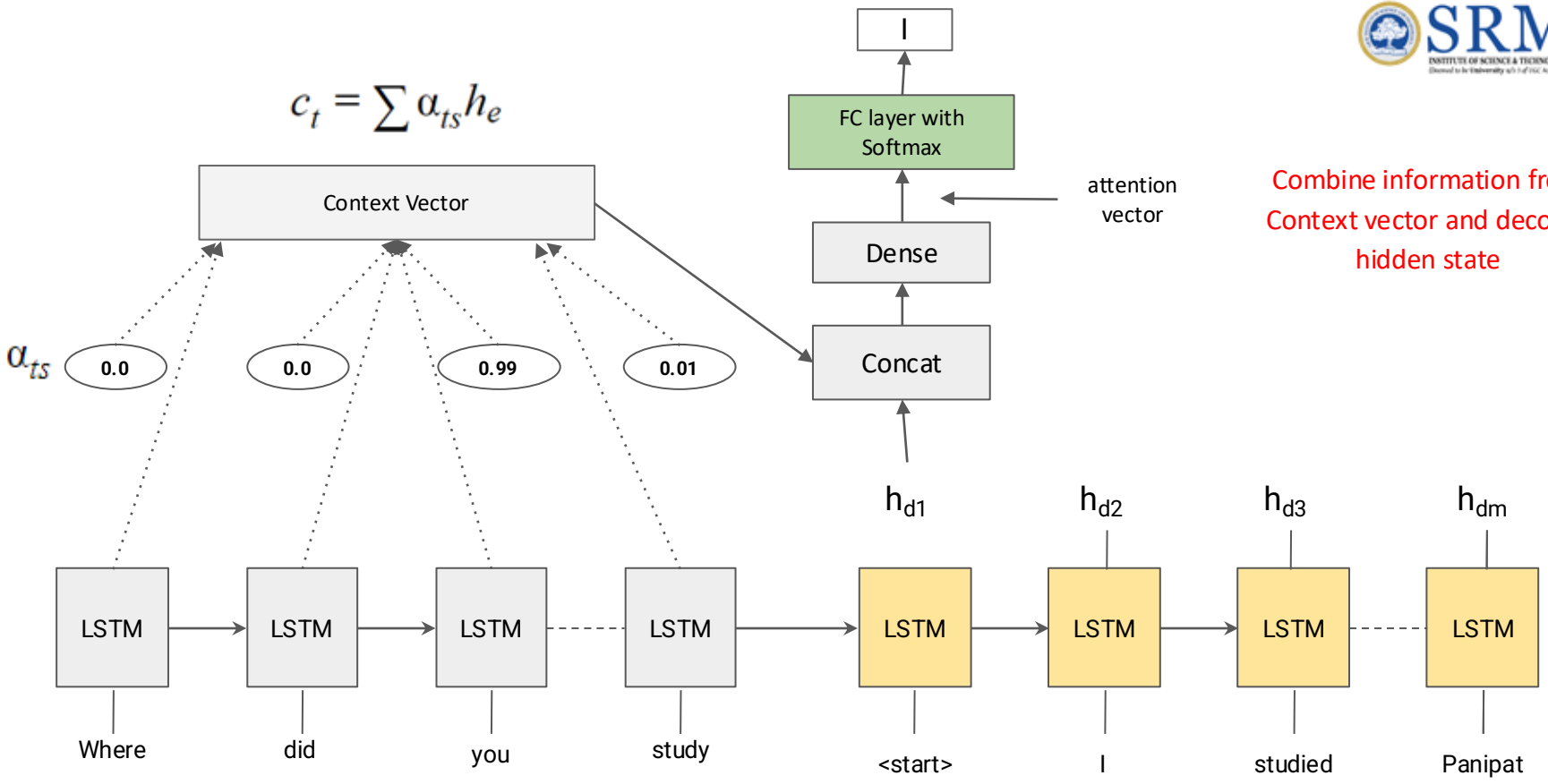
1. Multiply attention weights with encoder hidden states
2. Element-wise addition of multiplication output

How do we predict 1st word e.g '*I*' ?

We will use:

1. 1st hidden state of decoder
2. Context vector for 1st word

$$c_t = \sum \alpha_{ts} h_e$$

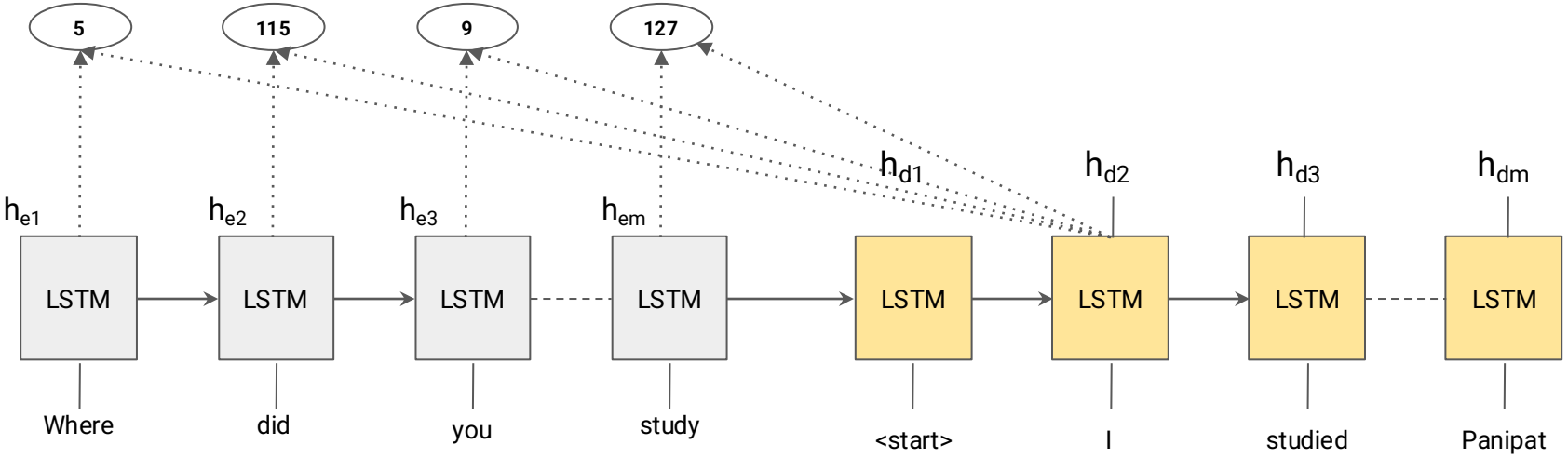


Combine information from Context vector and decoder hidden state

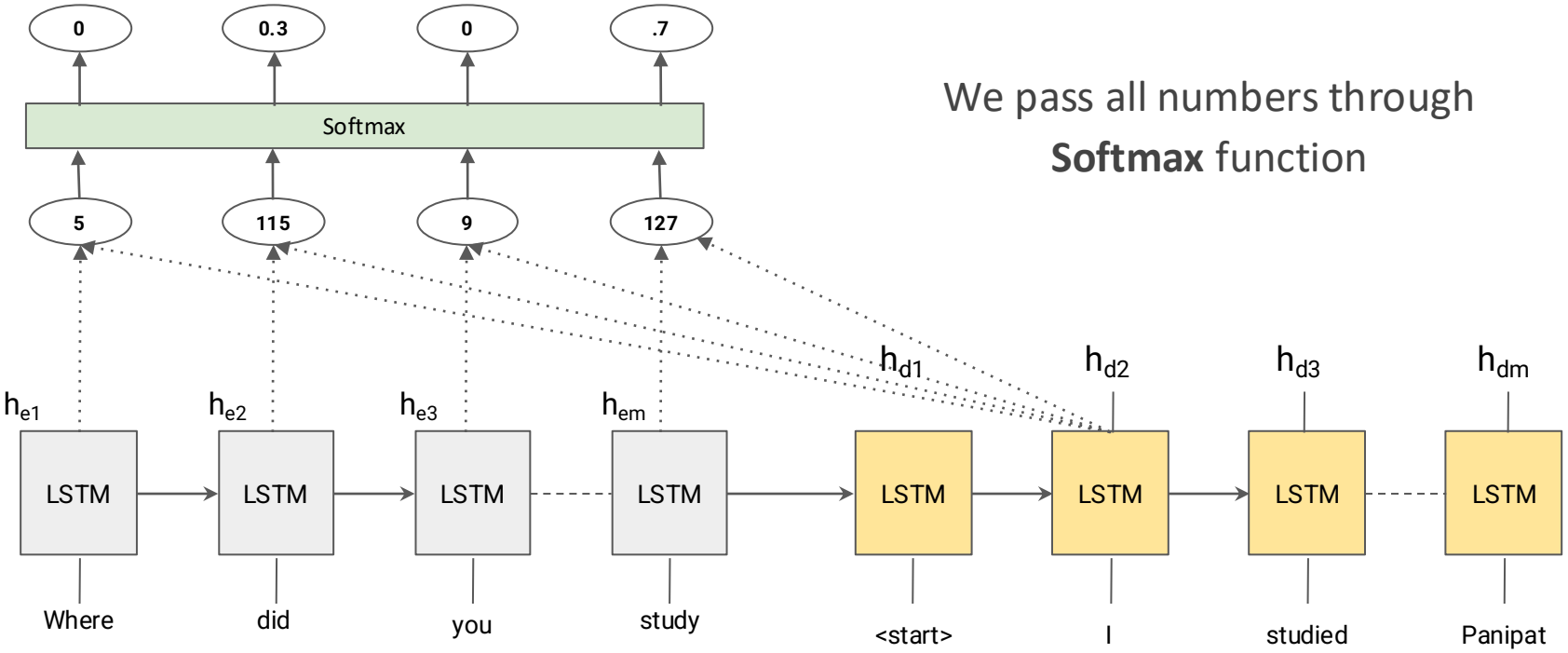
What's next?

We build attention weights for 2nd decoder output

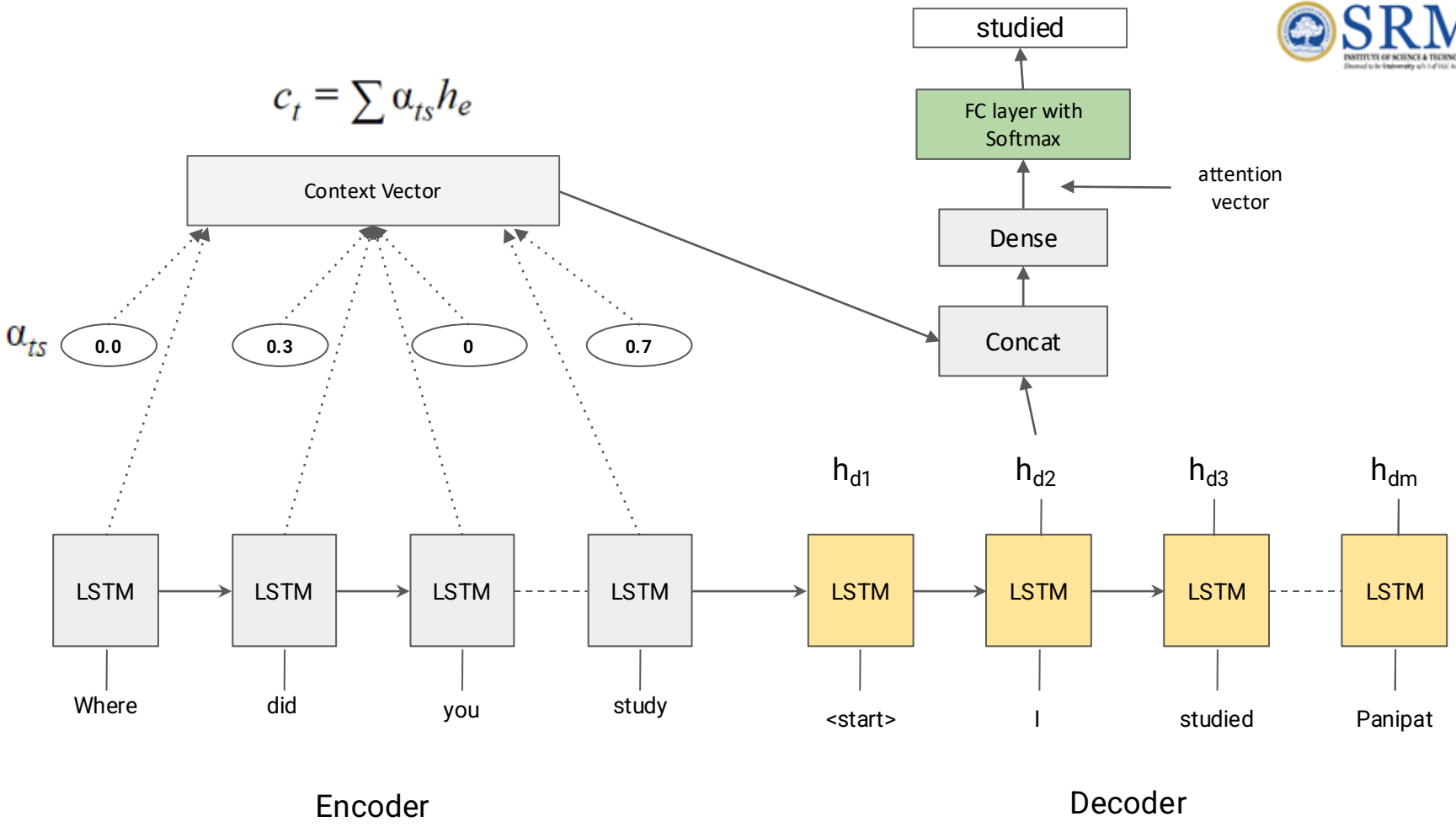
We do a '**dot**' product between each encoder states h_e with **2nd** decoder hidden state h_{d2}

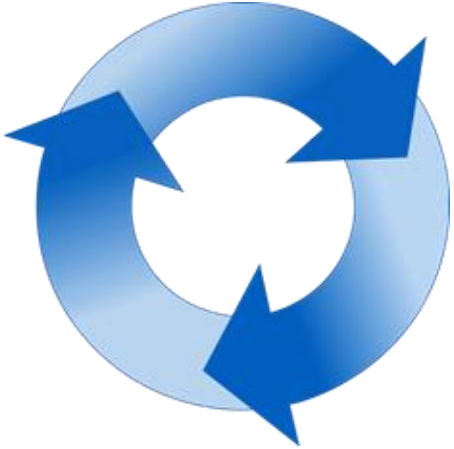


Attention weights to generate **studied**



We pass all numbers through **Softmax** function


$c_t = \sum \alpha_{ts} h_e$
studied
FC layer with Softmax
attention vector
Dense
Concat
Context Vector
α_{ts}
0.0
0.3
0
0.7
h_{d1}
h_{d2}
h_{d3}
h_{dm}
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
Where
did
you
study
<start>
I
studied
Panipat
Encoder
Decoder



Repeat these steps to predict all output words using attention!

Global Attention

Effective Approaches to Attention-based Neural Machine Translation
Minh-Thang Luong, et al.

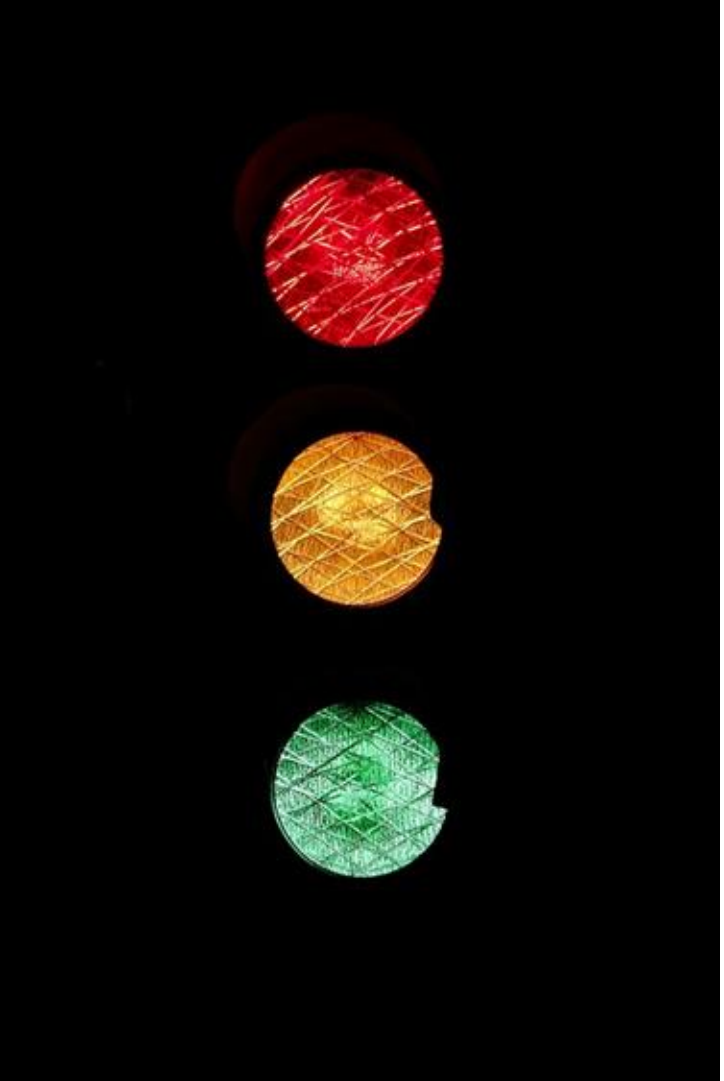
A woman is throwing a frisbee in a park.

A dog is standing on a hardwood floor.

A stop sign is on a road with a mountain in the background.

Seq2Seq model with Attention for image captioning

Show, attend and tell: Neural image caption generation with visual attention
K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R.S. Zemel, Y. Bengio.
arXiv preprint arXiv:1502.03044, Vol 2(3), pp. 5. CoRR. 2015.



Implementing Attention!

Data Preprocessing

No Change